

FOVL

Deeksha Varshney

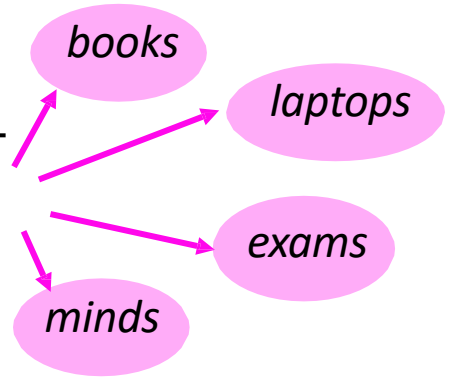
Language Models and RNNs

Content is taken from various sources

1. Language Modeling

- **Language Modeling** is the task of predicting what word comes next

the students opened their _____



- More formally: given a sequence of words $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$, compute the probability distribution of the next word $\boldsymbol{x}^{(t+1)}$:

$$P(\boldsymbol{x}^{(t+1)} | \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)})$$

where $\boldsymbol{x}^{(t+1)}$ can be any word in the vocabulary $V = \{\boldsymbol{w}_1, \ldots, \boldsymbol{w}_{|V|}\}$

- A system that does this is called a **Language Model**

Language Modeling

- You can also think of a Language Model as a system that assigns a probability to a piece of text

- For example, if we have some text $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$, then the probability of this text (according to the Language Model) is:

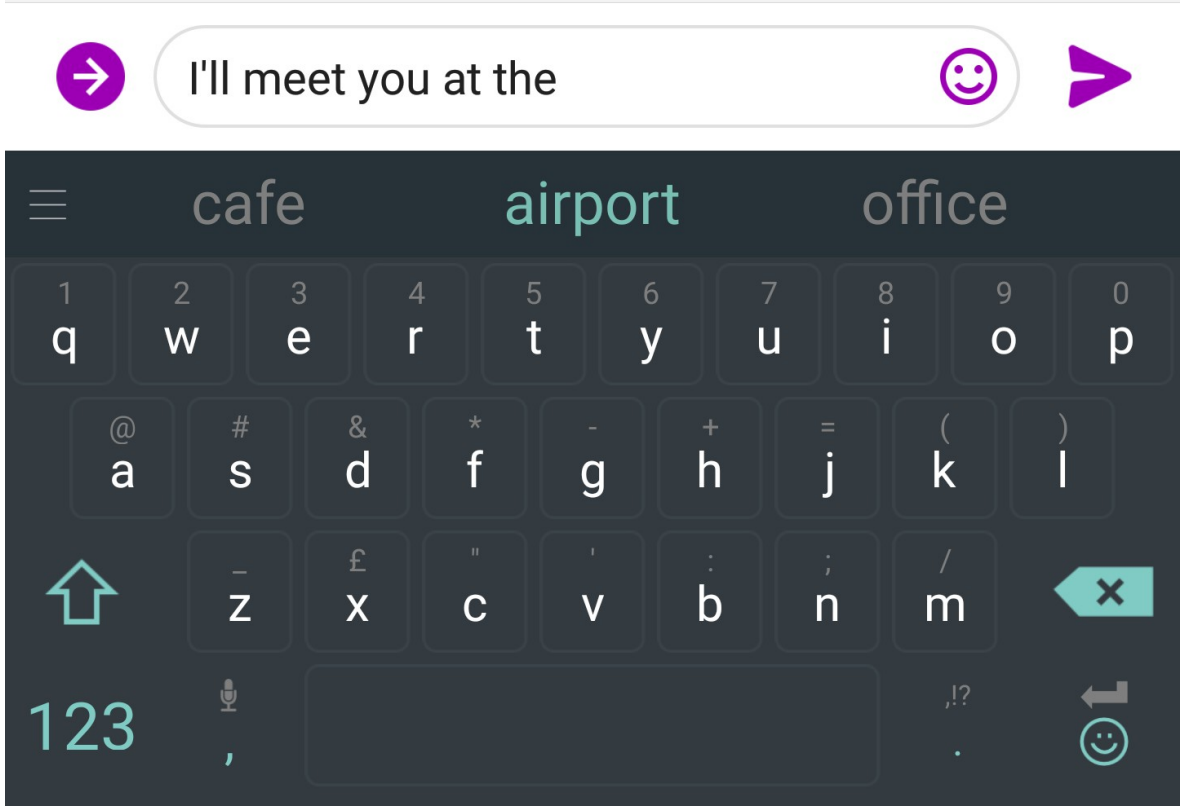
$$P(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}) = P(\boldsymbol{x}^{(1)}) \times P(\boldsymbol{x}^{(2)} | \boldsymbol{x}^{(1)}) \times \cdots \times P(\boldsymbol{x}^{(T)} | \boldsymbol{x}^{(T-1)}, \ldots, \boldsymbol{x}^{(1)})$$

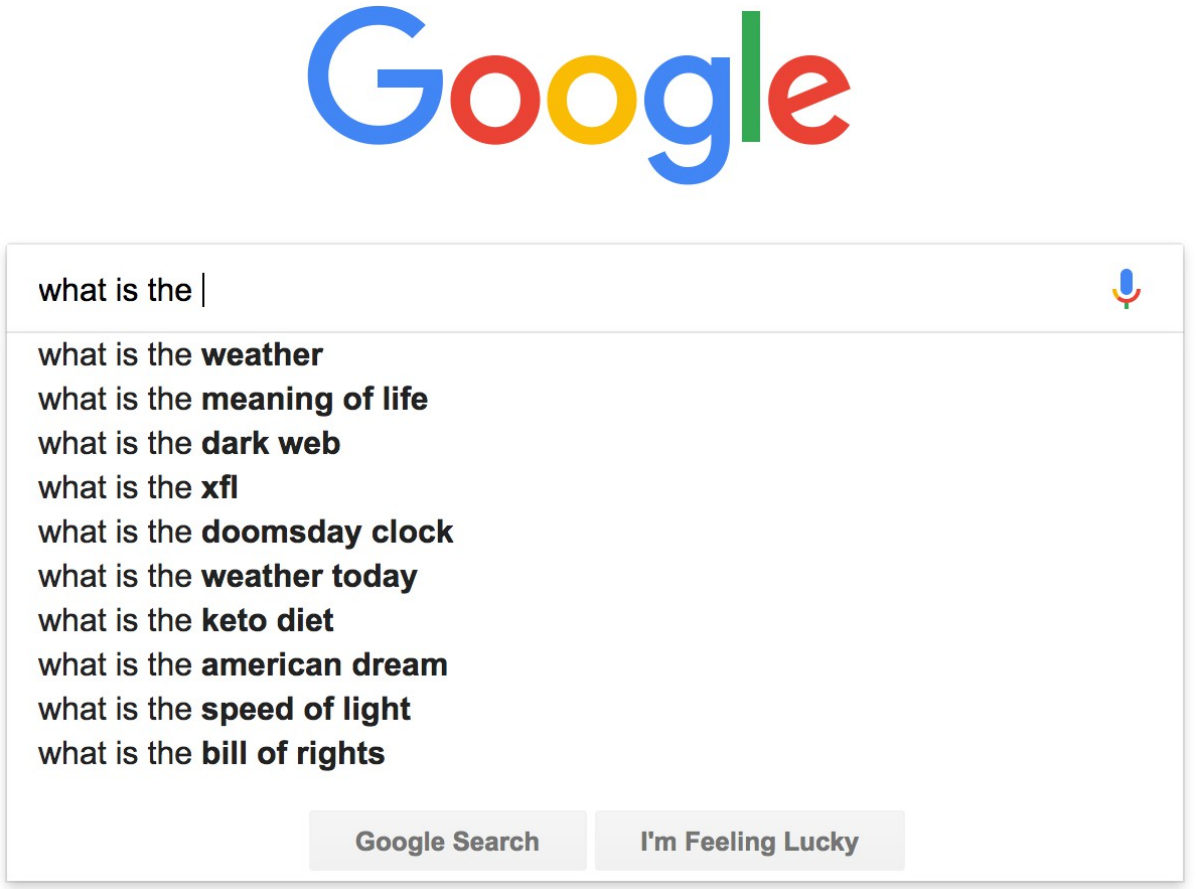
$$= \prod_{t=1}^{T} P(\boldsymbol{x}^{(t)} | \boldsymbol{x}^{(t-1)}, \ldots, \boldsymbol{x}^{(1)})$$

This is what our LM provides

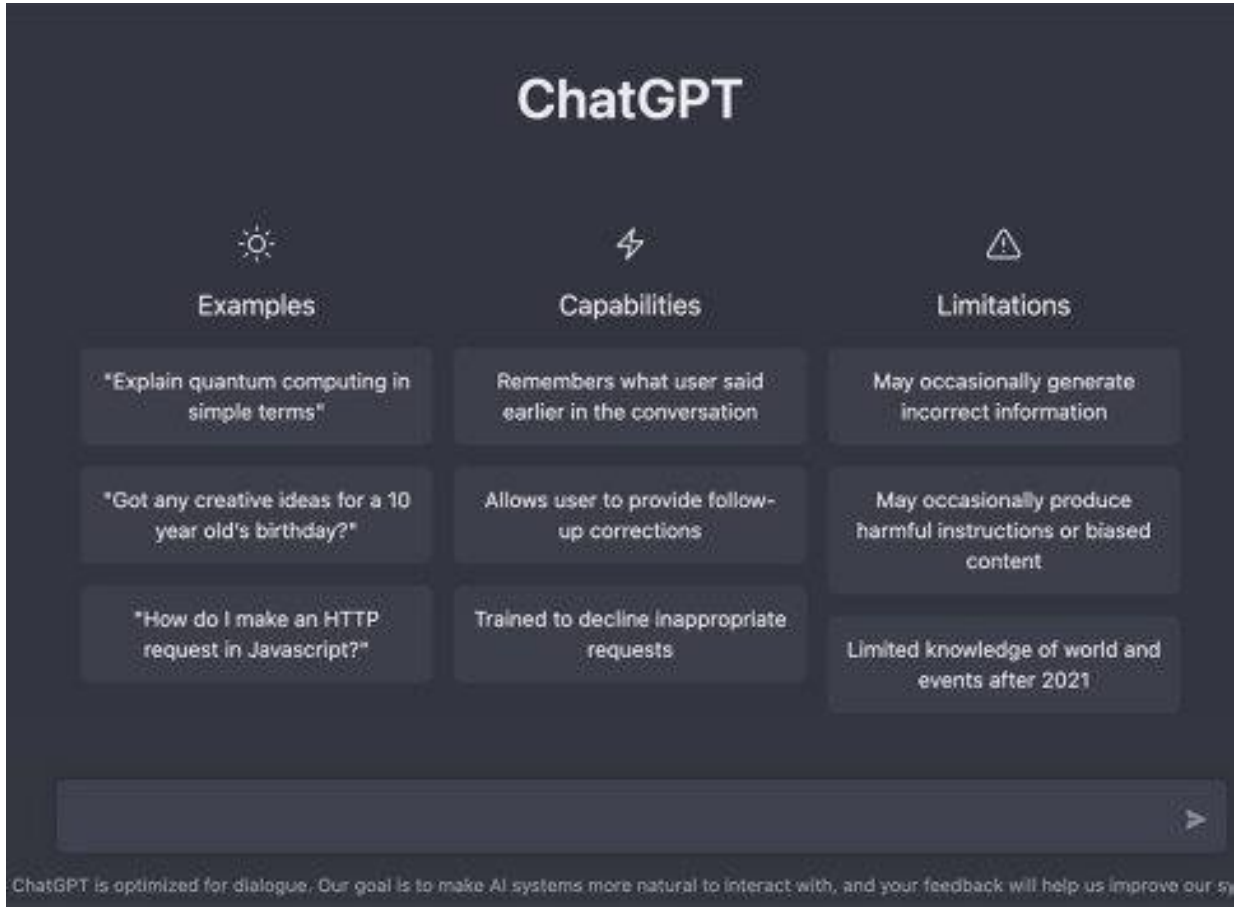
A language model assigns a probability to a sequence by multiplying conditional probabilities using the chain rule.

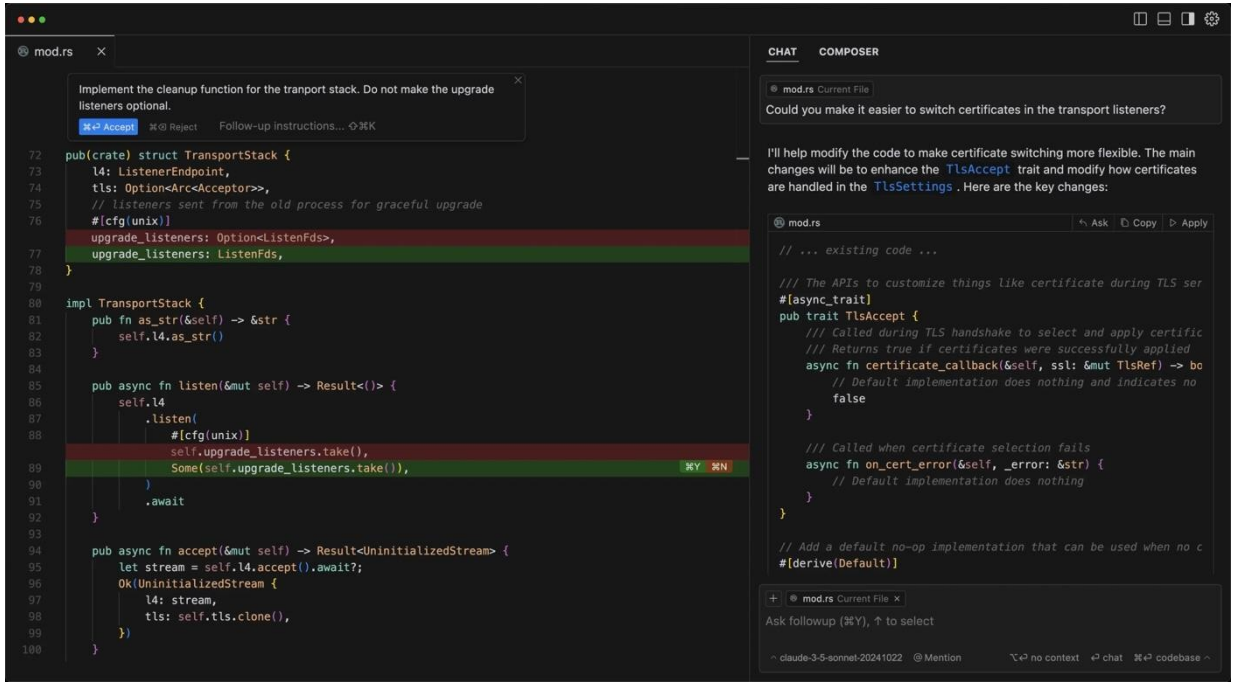
You use Language Models every day!





You use Language Models every day!





Why should we care about Language Modeling?

- Language Modeling is a benchmark task that helps us measure our progress on predicting language use
- Language Modeling is a subcomponent of many NLP tasks, especially those involving generating text or estimating the probability of text:
    - Predictive typing
    - Speech recognition
    - Handwriting recognition
    - Spelling/grammar correction
    - Authorship identification
    - Machine translation
    - Summarization
    - Dialogue
    - etc.
- Everything else in NLP has been rebuilt upon Language Modeling: ChatGPT is an LM!

What can you do with next-word prediction?

- A sufficiently strong (!) language model can do many, many things

Stanford University is located in _____, California. [Trivia]

I put ____fork down on the table. [syntax]

The woman walked across the street, checking for traffic over _____shoulder. [coreference]

I went to the ocean to see the fish, turtles, seals, and _____. [lexical semantics/topic]

Overall, the value I got from the two hours watching it was the sum total of the popcorn and the drink. The movie was ____. [sentiment]

Iroh went into the kitchen to make some tea. Standing next to Iroh, Zuko pondered his destiny. Zuko left the _____. [some reasoning – this is harder]

I was thinking about the sequence that goes 1, 1, 2, 3, 5, 8, 13, 21, _____ [some basic arithmetic]

n-gram Language Models

the students opened their _____

- **Question**: How to learn a Language Model?
- **Answer** (pre- Deep Learning): learn an *n*-gram Language Model!

- **Definition:** An *n*-gram is a chunk of *n* consecutive words.
  - unigrams: “the”, “students”, “opened”, ”their”
  - bigrams: “the students”, “students opened”, “opened their”
  - trigrams: “the students opened”, “students opened their”
  - four-grams: “the students opened their”

- **Idea:** Collect statistics about how frequent different n-grams are and use these to predict next word.

n-gram Language Models

- First we make a Markov assumption: $x^{(t+1)}$ depends only on the preceding *n*-1 words

n-1 words

$$P(\boldsymbol{x}^{(t+1)}|\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(1)}) = P(\boldsymbol{x}^{(t+1)}|\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)})$$

(assumption)

prob of a n-gram

prob of a (n-1)-gram

$$= \frac{P(\boldsymbol{x}^{(t+1)},\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)})}{P(\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)})}$$

(definition of conditional prob)

- **Question:** How do we get these *n*-gram and (*n*-1)-gram probabilities?
- **Answer:** By counting them in some large corpus of text!

$$\approx \frac{\text{count}(\boldsymbol{x}^{(t+1)},\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)})}{\text{count}(\boldsymbol{x}^{(t)},\ldots,\boldsymbol{x}^{(t-n+2)})}$$

(statistical approximation)

n-gram Language Models: Example

Suppose we are learning a 4-gram Language Model.

~~*as the proctor started the clock, the*~~ *students opened their* _____

discard

You **discard everything before the last 3 words**

condition on this

$$P(\boldsymbol{w}|\text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

For example, suppose that in the corpus:

- "students opened their" occurred 1000 times
- "students opened their books" occurred 400 times
  - → P(books | students opened their) = 0.4
- "students opened their exams" occurred 100 times
  - → P(exams | students opened their) = 0.1

Should we have discarded the "proctor" context?

Sparsity Problems with n-gram Language Models

Why Is This Bad?
Because:

- The sentence might be perfectly valid
- The model says it is impossible
- Any probability calculation involving this becomes zero

Sparsity Problem 1

Problem: What if *"students opened their w"* never occurred in data? Then w has probability 0!

(Partial) Solution: Add small δ to the count for every $w \in V$. This is called *smoothing*.

$$P(\boldsymbol{w}|\text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

Why Is This Worse?
Because now the model cannot even guess.
There is no fallback unless extra tricks are added.

Sparsity Problem 2

Problem: What if *"students opened their"* never occurred in data? Then we can't calculate probability for *any* w!

(Partial) Solution: Just condition on *"opened their"* instead. This is called *backoff*.

Note: Increasing n makes sparsity problems *worse*. Typically, we can't have n bigger than 5.

Storage Problems with n-gram Language Models

Storage: Need to store count for all *n*-grams you saw in the corpus.

$$P(\boldsymbol{w}|\text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

Increasing *n* or increasing corpus increases model size!

n-gram Language Models in practice

- You can build a simple trigram Language Model over a 1.7 million word corpus (Reuters) in a few seconds on your laptop*

Business and financial news

today the _____

get probability distribution

company	0.153
bank	0.153
price	0.077
italian	0.039
emirate	0.039

Otherwise, seems reasonable!

Sparsity problem: not much granularity in the probability distribution

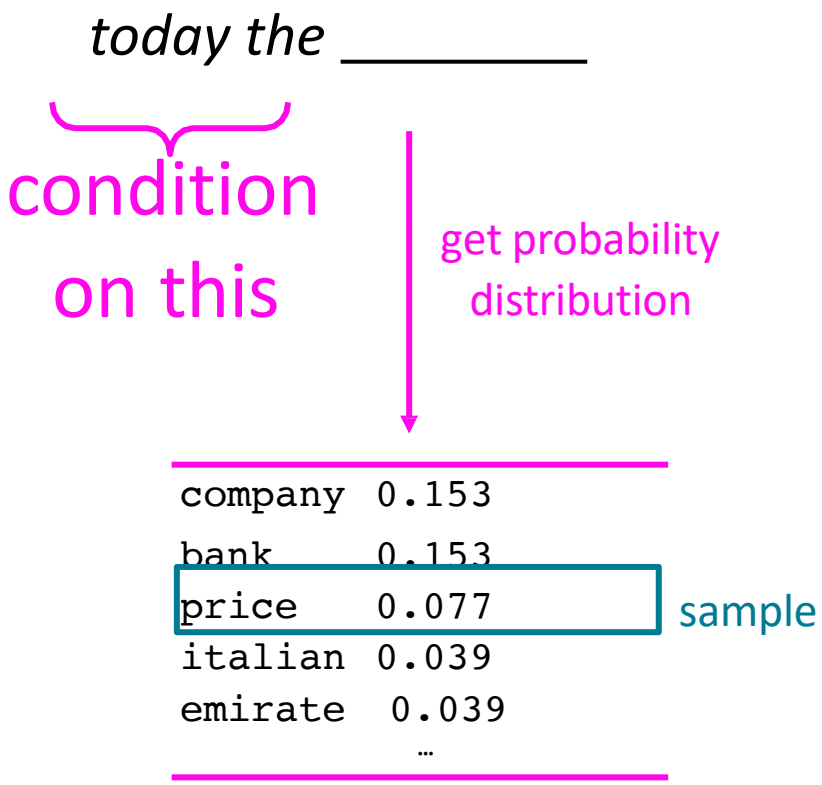
Ideally, the probability distribution should be:

- Smooth
- Fine-grained
- Able to distinguish between many reasonable next words

* Try for yourself: https://nlpforhackers.io/language-models/

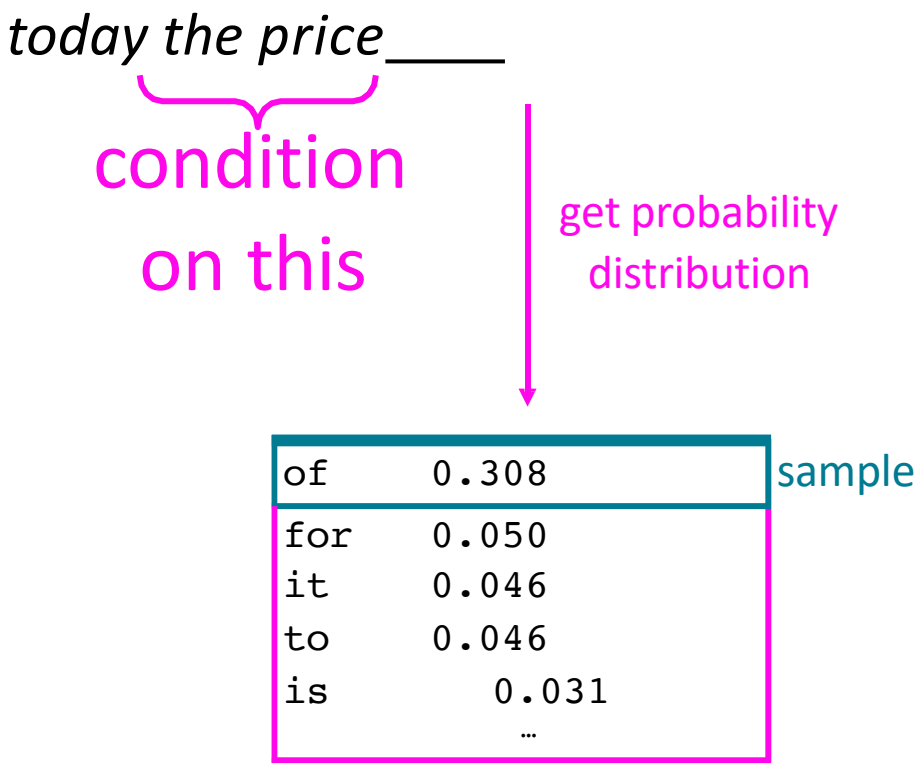
Generating text with a n-gram Language Model

You can also use a Language Model to generate text



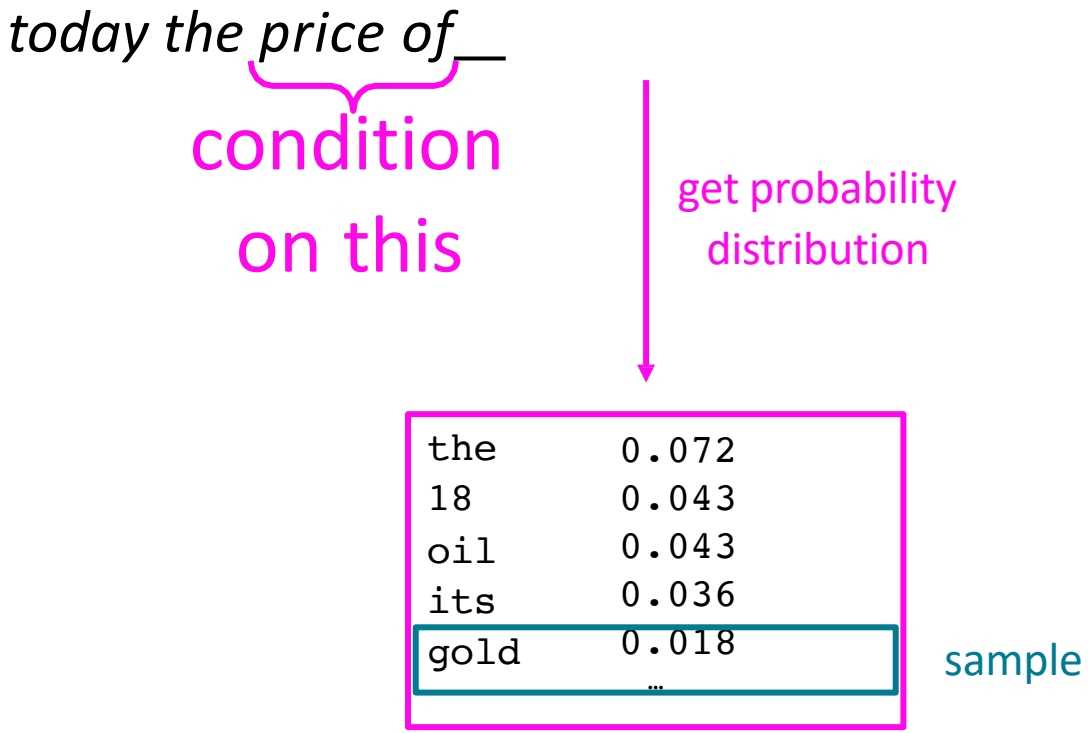
Generating text with a n-gram Language Model

You can also use a Language Model to generate text



Generating text with a n-gram Language Model

You can also use a Language Model to generate text



Generating text with a n-gram Language Model

You can also use a Language Model to generate text

today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .

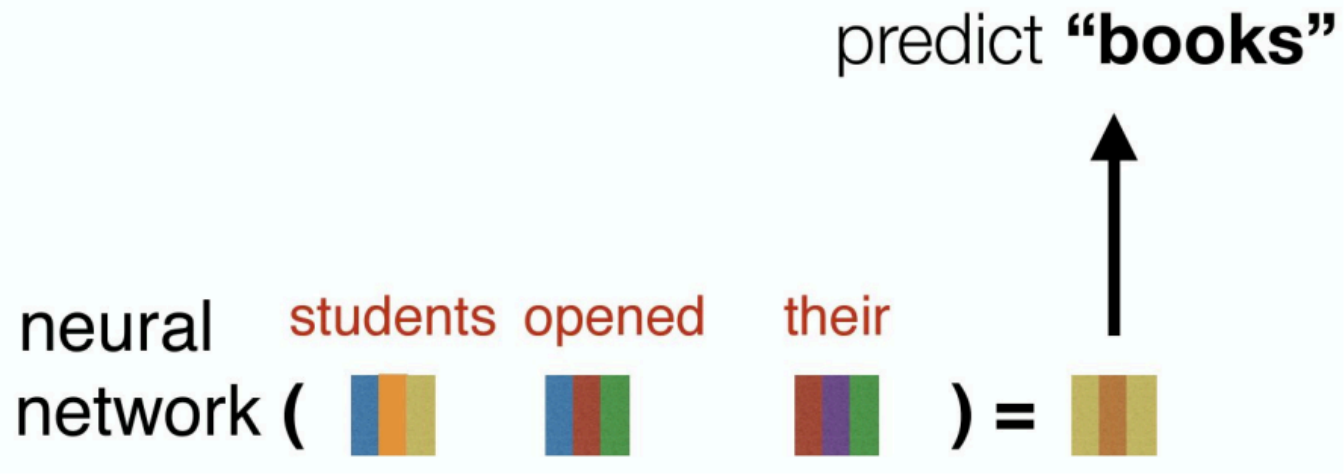
Surprisingly grammatical!

…but **incoherent.** We need to consider more than three words at a time if we want to model language well.

But increasing *n* worsens sparsity problem, and increases model size…

- Neural networks compose word embeddings into vectors for phrases, sentences, and documents

- Predict the next word from composed prefix representation





How does this happen?

Let's say our output vocabulary consists of just four words: "books", "houses", "lamps", and "stamps".

books houses lamps stamps

<0.6, 0.2, 0.1, 0.1>

We want to get a probability distribution over these four words

$\boldsymbol{x}$ = <-2.3, 0.9, 5.4>



Here's an example 3-d prefix vector

How does this happen?

W is a *weight matrix*. It contains *parameters* that we can *update* to control the final probability distribution of the next word

$$\mathbf{W} = \begin{Bmatrix} 1.2, & -0.3, & 0.9 \\ 0.2, & 0.4, & -2.2 \\ 8.9, & -1.9, & 6.5 \\ 4.5, & 2.2, & -0.1 \end{Bmatrix}$$

first, we'll project our 3-d prefix representation to 4-d with a matrix-vector product

$$\boldsymbol{x} = \langle -2.3, 0.9, 5.4 \rangle$$

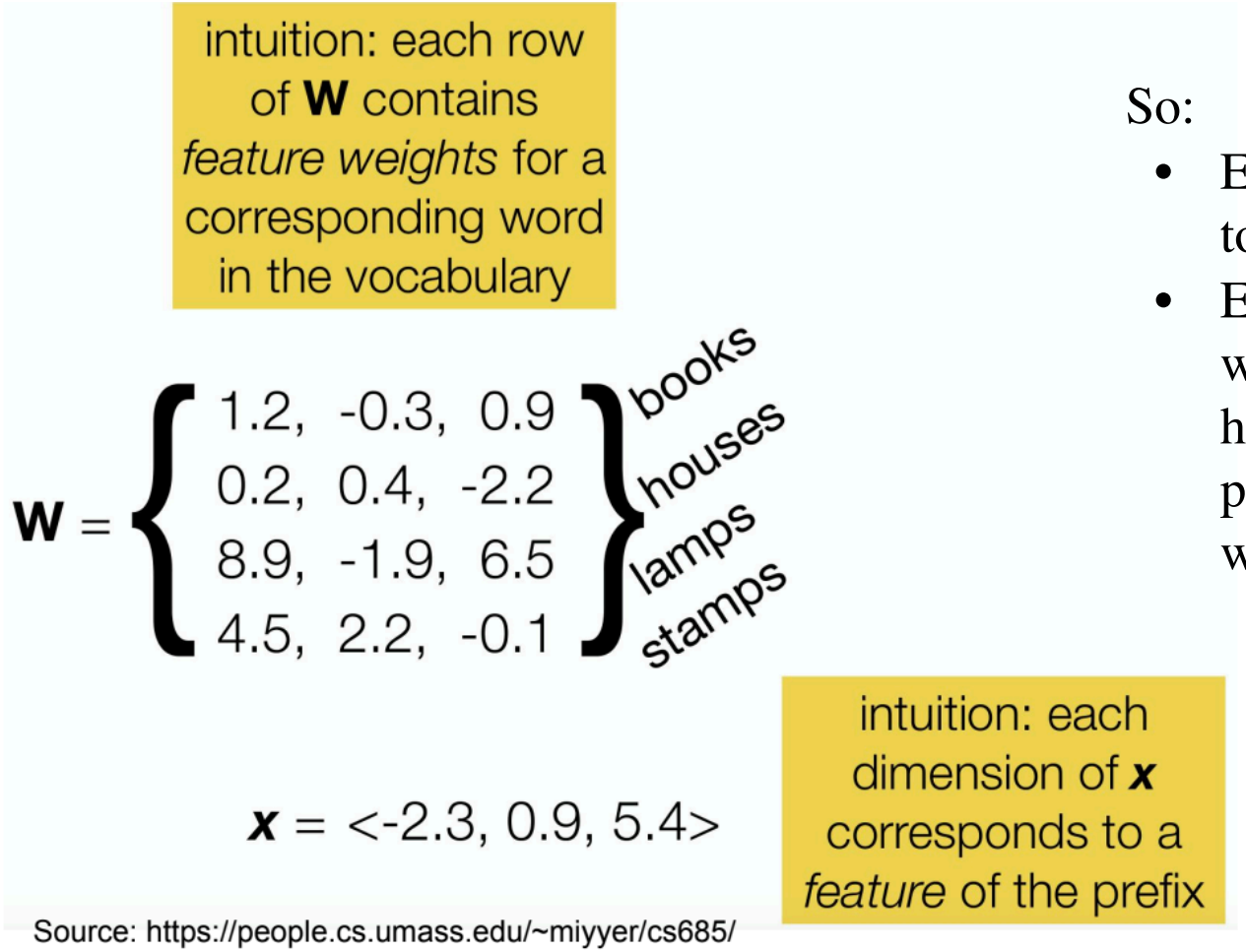


Here's an example 3-d prefix vector

Why this shape?

- Input vector has 3 dimensions
- Output vocabulary has 4 words

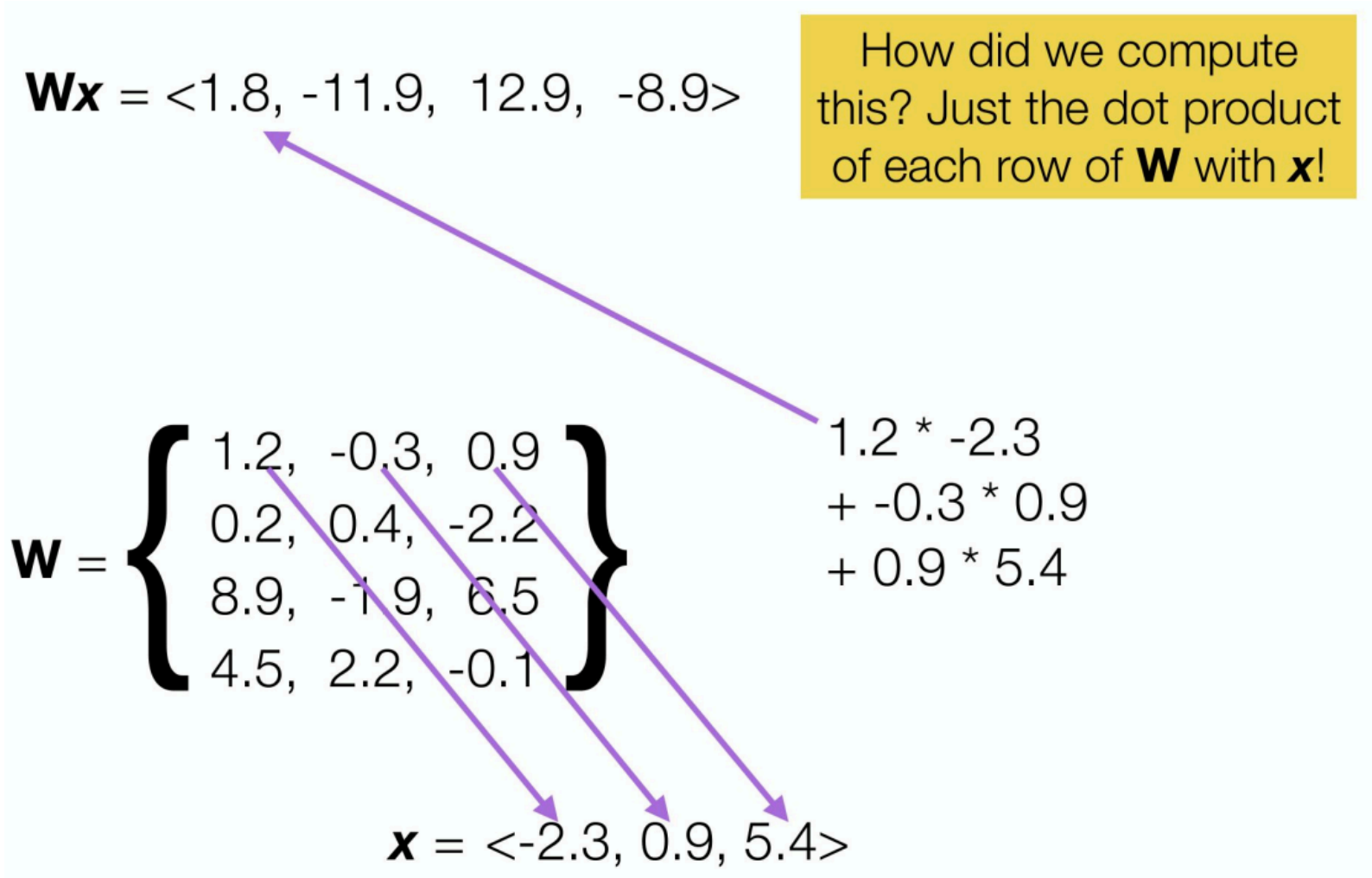
How does this happen?



So:

- Each row corresponds to one output word
- Each row stores weights that score how compatible the prefix is with that word

How does this happen?



How does this happen?

Okay, so how do we go from this 4-d vector to a probability distribution?

We'll use the **softmax** function!

$$\text{softmax}(x) = \frac{e^{x}}{\sum_{j} e^{x_j}}$$

Wx = <1.8, -11.9, 12.9, -8.9>

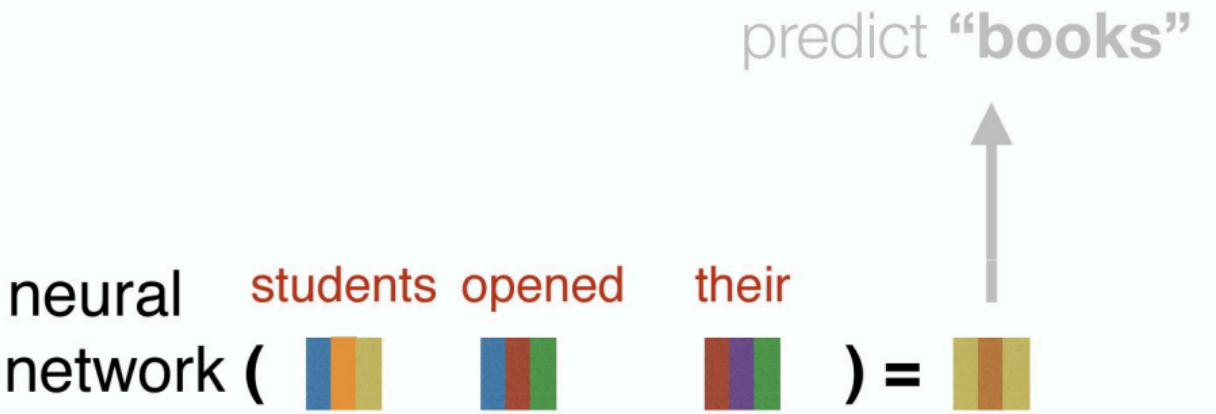
Wx = <1.8, -1.9, 2.9, -0.9>

softmax(Wx) = <0.24, 0.006, 0.73, 0.02>

books houses lamps stamps

Computing prefix representation

Now that we know how to predict **“books”**, let’s focus on how to compute the prefix representation ***x*** in the first place!



How does this happen?

Composition functions

input: sequence of word embeddings corresponding to the tokens of a given prefix
output: single vector

- Element-wise functions
- e.g., just sum up all of the word embeddings!
- Concatenation
- Feed-forward neural networks
- Convolutional neural networks
- Recurrent neural networks
- Transformers

Consider a model that predicts the output using the equation $\hat{y} = \tanh(wx + b)$, where w is the weight and b is the bias. The model is trained using the dataset (1, 0.35), (3, 0.60), and (5, 0.75). The total cost is defined as the sum of squared errors across all training examples.

The following derivative rules are provided:

- Derivative of z^2 with respect to z is $2z$.
- Derivative of $\tanh(z)$ is $1 - \tanh^2(z)$.
- Derivative of $(wx + b)$ with respect to w is x.

(a) (2 marks) Considering only the first training example ($x = 1$, $y = 0.35$), derive the contribution of this example to the partial derivative of the cost with respect to w. Show the steps using the chain rule.

(b) (2 marks) Suppose the partial derivative of the total cost with respect to w is negative. If the weight is updated using $w \leftarrow w - \eta(\partial Cost/\partial w)$, where η is a positive learning rate, will the weight increase or decrease? Explain why.

(c) (3 marks) Suppose the current values are $w = 0.2$ and $b = 0.1$. For the first training example, calculate the predicted output and the contribution of this example to the gradient with respect to w. Use $\tanh(0.3) \approx 0.2913$.

(d) (3 marks) Explain why gradient descent subtracts the gradient during parameter updates. Does a negative gradient always guarantee that the cost will decrease after an update, regardless of the learning rate? Justify your answer.

A Skip-gram Word2Vec model is trained with the following configuration:

- Vocabulary size $= 50{,}000$
- Embedding dimension $= 200$
- Context window size $= 2$ on each side (maximum 4 context words)
- Negative sampling with $k = 10$

For one target word occurrence, assume the full context window is used. How many sigmoid computations are performed during training for this target word occurrence?

(a) 4
(b) 14
(c) 44
(d) 400

2. Consider a Skip-Gram model with vocabulary size $V = 4$ and embedding dimension $d = 2$. Let the input (center word) embedding matrix be

$$W = \begin{bmatrix} 0.2 & -0.1 & 0.4 & 0.0 \\ -0.3 & 0.5 & 0.1 & -0.2 \end{bmatrix} \in \mathbb{R}^{2\times 4},$$

and the output embedding matrix be

$$W' = \begin{bmatrix} 0.1 & -0.2 \\ 0.0 & 0.3 \\ -0.4 & 0.2 \\ 0.2 & 0.1 \end{bmatrix} \in \mathbb{R}^{4\times 2}.$$

Suppose the center word is word 3, represented as the one-hot vector $x = [0\ 0\ 1\ 0]^T$, and the true context word is word 2, represented as $y = [0\ 1\ 0\ 0]^T$.

(a) First compute the hidden representation $v_c = Wx$ and explain why this operation corresponds to selecting a column of W. **[2 marks]**

(b) Next compute the score vector $u = W'v_c$. Then apply the softmax function

$$\hat{y}_i = \frac{e^{u_i}}{\sum_{j=1}^{V} e^{u_j}}$$

to obtain the predicted probability distribution $\hat{y}$. **[2 marks]**

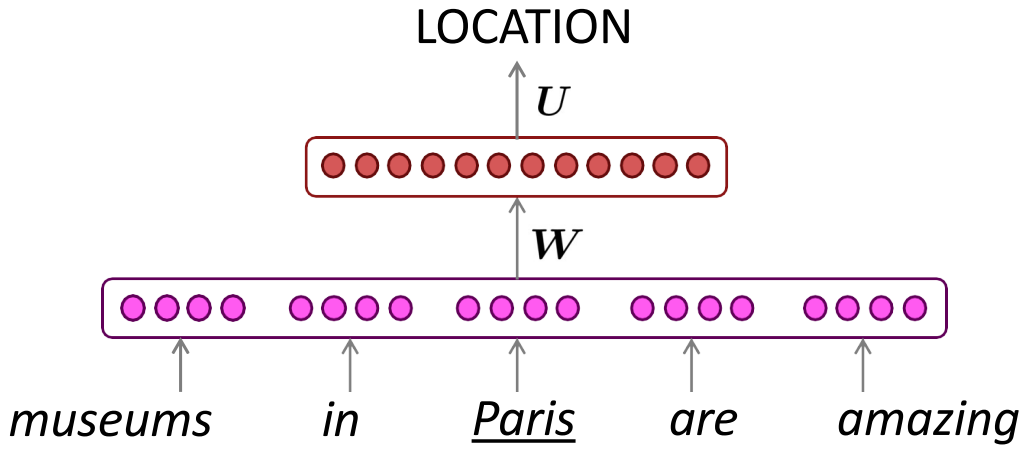
(c) Identify which word has the highest predicted probability. **[1 marks]**

(d) Finally, state clearly what the final embedding vector (representation) is for the center word and explain why this vector is considered the learned representation of that word in the Skip-Gram model. Why is this embedding static? **[2 marks]**

How to build a *neural* language model?

- Recall the Language Modeling task:
  - Input: sequence of words $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$
  - Output: prob. dist. of the next word $P(\boldsymbol{x}^{(t+1)} | \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)})$

- How about a window-based neural model?
  - We can apply this to Named Entity Recognition:



A fixed-window neural Language Model

output distribution

$$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{U}\boldsymbol{h} + \boldsymbol{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

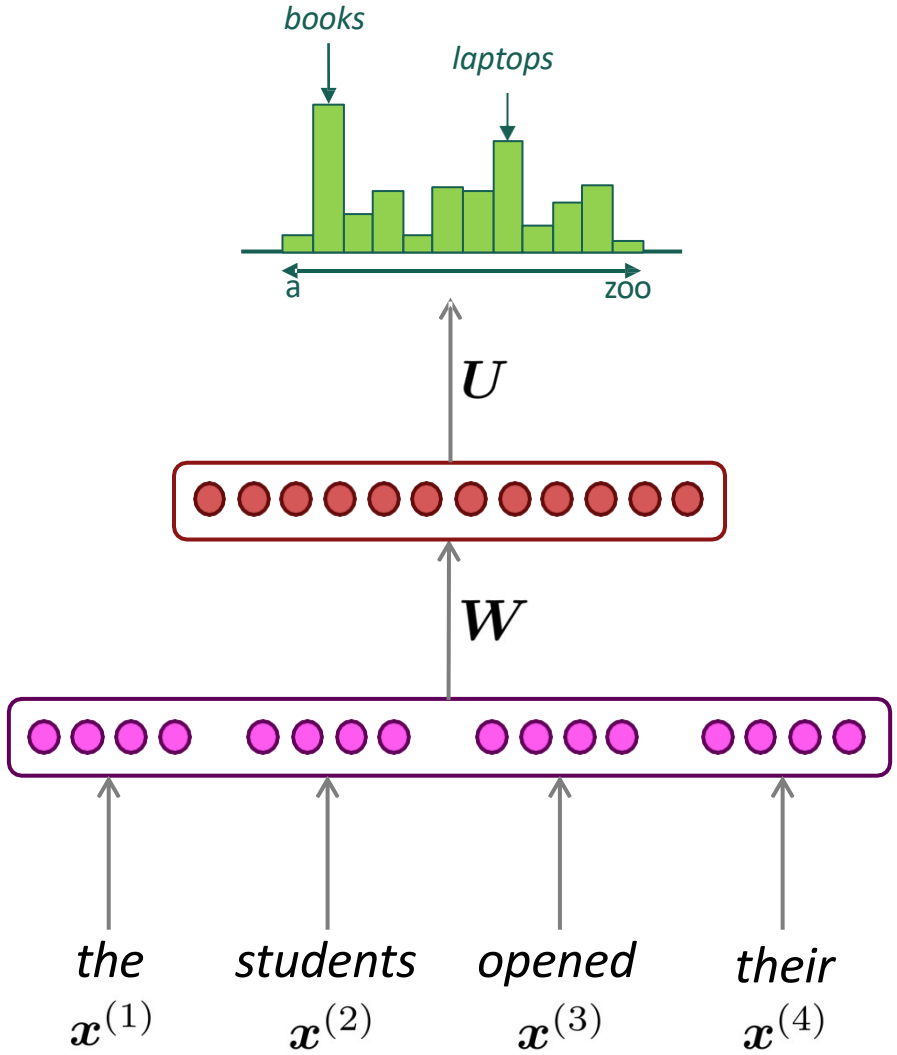
$$\boldsymbol{h} = f(\boldsymbol{W}\boldsymbol{e} + \boldsymbol{b}_1)$$

concatenated word embeddings

$$\boldsymbol{e} = [\boldsymbol{e}^{(1)}; \boldsymbol{e}^{(2)}; \boldsymbol{e}^{(3)}; \boldsymbol{e}^{(4)}]$$

words / one-hot vectors

$$\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(3)}, \boldsymbol{x}^{(4)}$$



A fixed-window neural Language Model

Approximately: Y. Bengio, et al. (2000/2003): A Neural Probabilistic Language Model

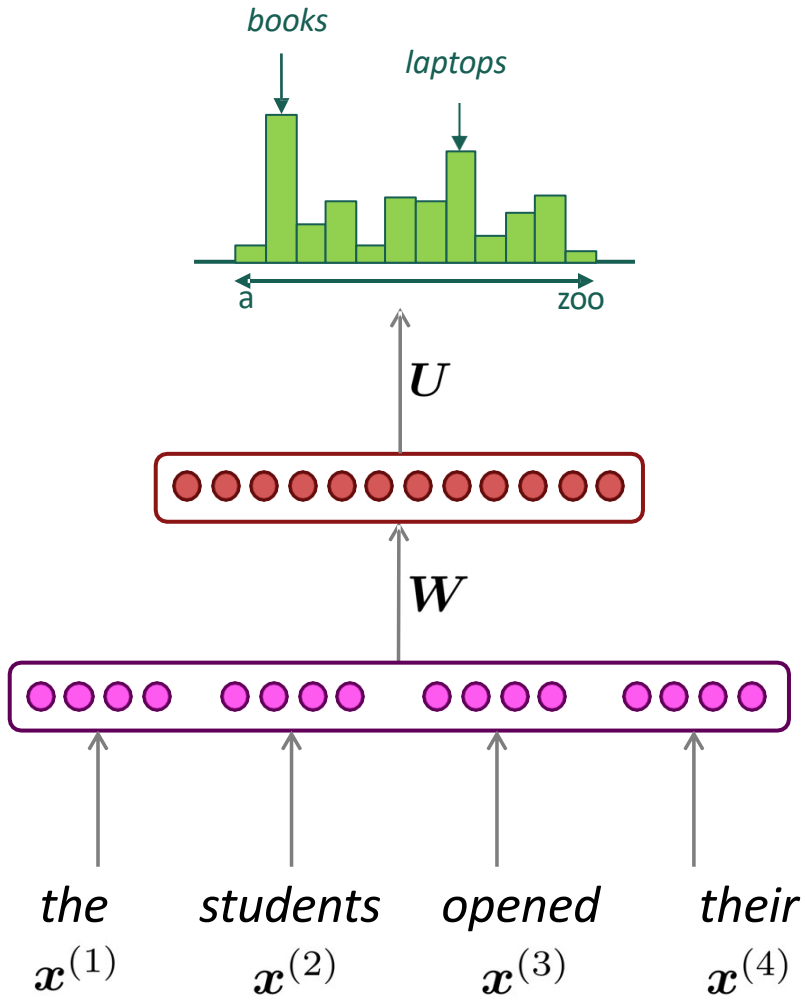
Improvements over *n*-gram LM:

- No sparsity problem
- Don't need to store all observed *n*-grams

Remaining **problems**:

- Fixed window is too small
- Enlarging window enlarges W
- Window can never be large enough!
- $x^{(1)}$ and $x^{(2)}$ are multiplied by completely different weights in W. No symmetry in how the inputs are processed.

We need a neural architecture that can process *any length input*



What Is the Problem?

Example:

Suppose the word "**students**" appears:

- At position x^2 in one sentence
- At position x^3 in another sentence

Then:

- At position x^2 → it uses weights W2
- At position x^3 → it uses weights W3

So the model treats the **same word differently depending on position**.

Why Is This Bad?

1. No Parameter Sharing

The model must learn:

- "students in position 2"
- "students in position 3"
- "students in position 4"

as **separate patterns**.

This wastes data and learning capacity.

2. No Symmetry

Language does not care about absolute window positions.

Example:

Both contexts should give similar meaning:

"the students opened their"

"yesterday students opened their"

But the fixed-window model sees:

- students at different positions
- different weight matrices
- different internal representations

This is what the slide means by:

No symmetry in how inputs are processed

3. **Poor Generalization**

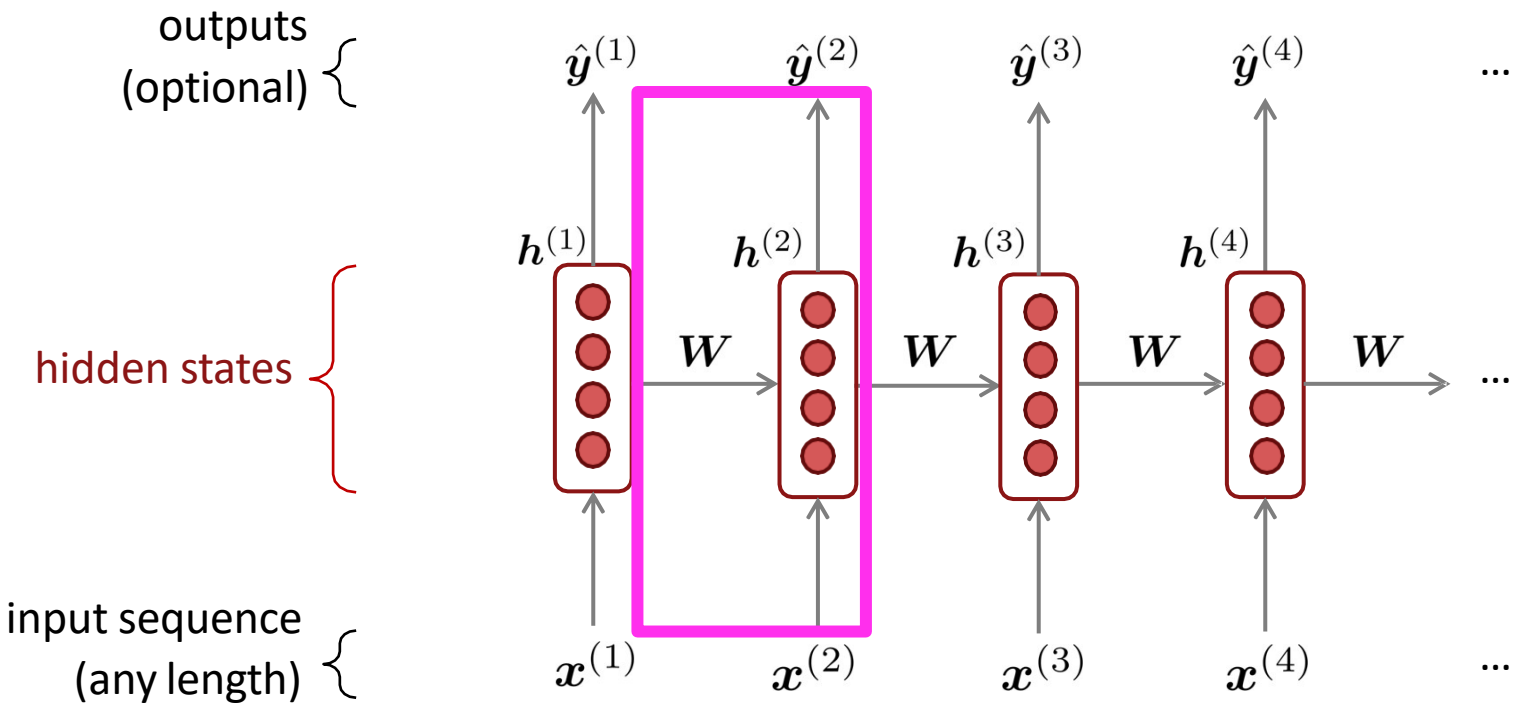
Because weights are not shared:

- Learning does not transfer across positions
- Model needs more data
- Training is inefficient

2. Recurrent Neural Networks (RNN)

A family of neural architectures

Core idea: Apply the same weights W *repeatedly*



A Simple RNN Language Model

output distribution

$$\hat{\boldsymbol{y}}^{(t)} = \text{softmax}\left(\boldsymbol{U}\boldsymbol{h}^{(t)} + \boldsymbol{b}_2\right) \in \mathbb{R}^{|V|}$$

hidden states

$$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{W}_h\boldsymbol{h}^{(t-1)} + \boldsymbol{W}_e\boldsymbol{e}^{(t)} + \boldsymbol{b}_1\right)$$

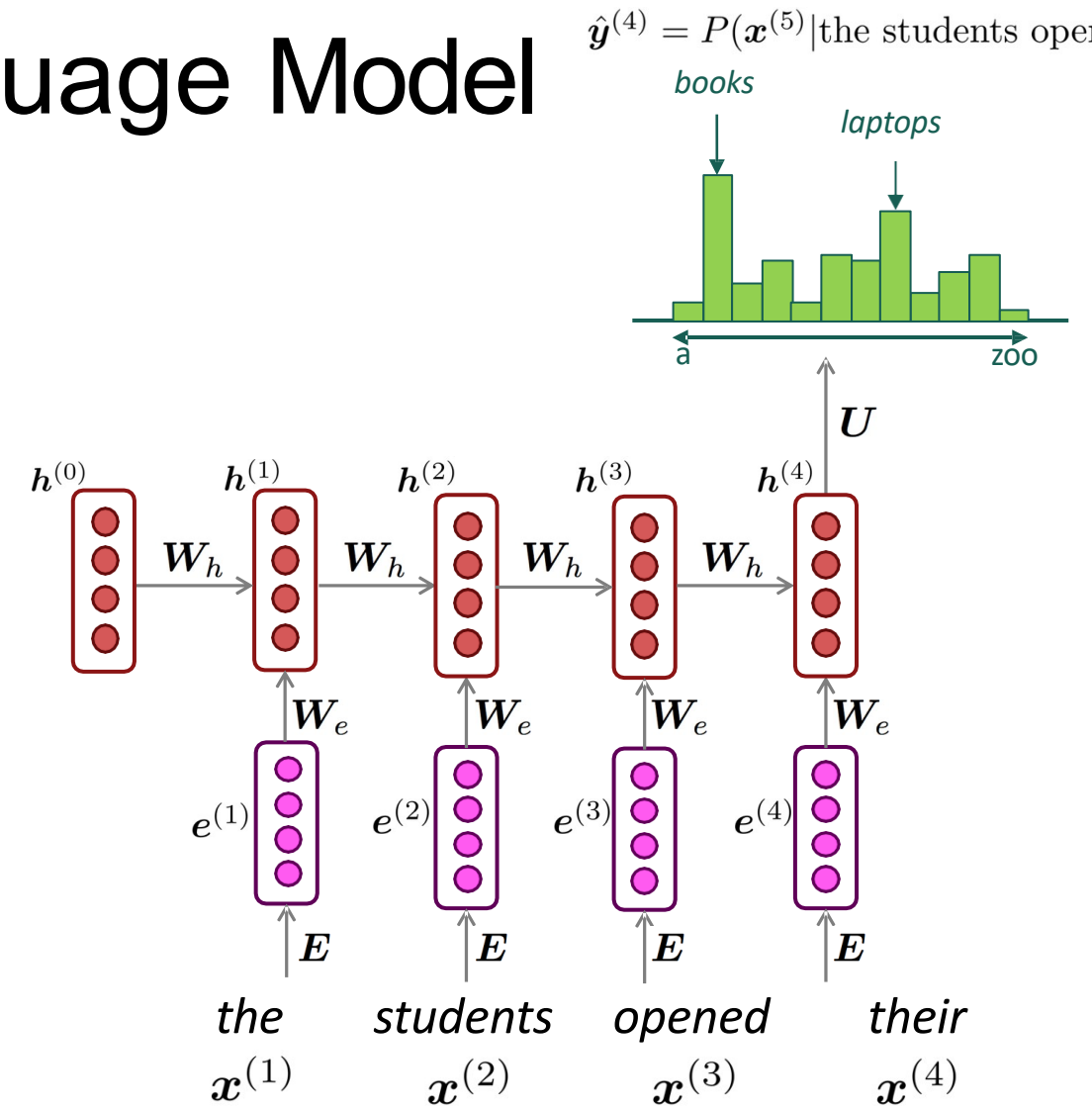
$\boldsymbol{h}^{(0)}$ is the initial hidden state

word embeddings

$$\boldsymbol{e}^{(t)} = \boldsymbol{E}\boldsymbol{x}^{(t)}$$

words / one-hot vectors

$$\boldsymbol{x}^{(t)} \in \mathbb{R}^{|V|}$$



Note: *this input sequence could be much longer now!*

Hidden state update:

$$h^{(t)} = \tanh(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_1)$$

Output prediction:

$$\hat{y}^{(t)} = softmax(W_{oh}h^{(t)} + b_2)$$

RNN Language Models

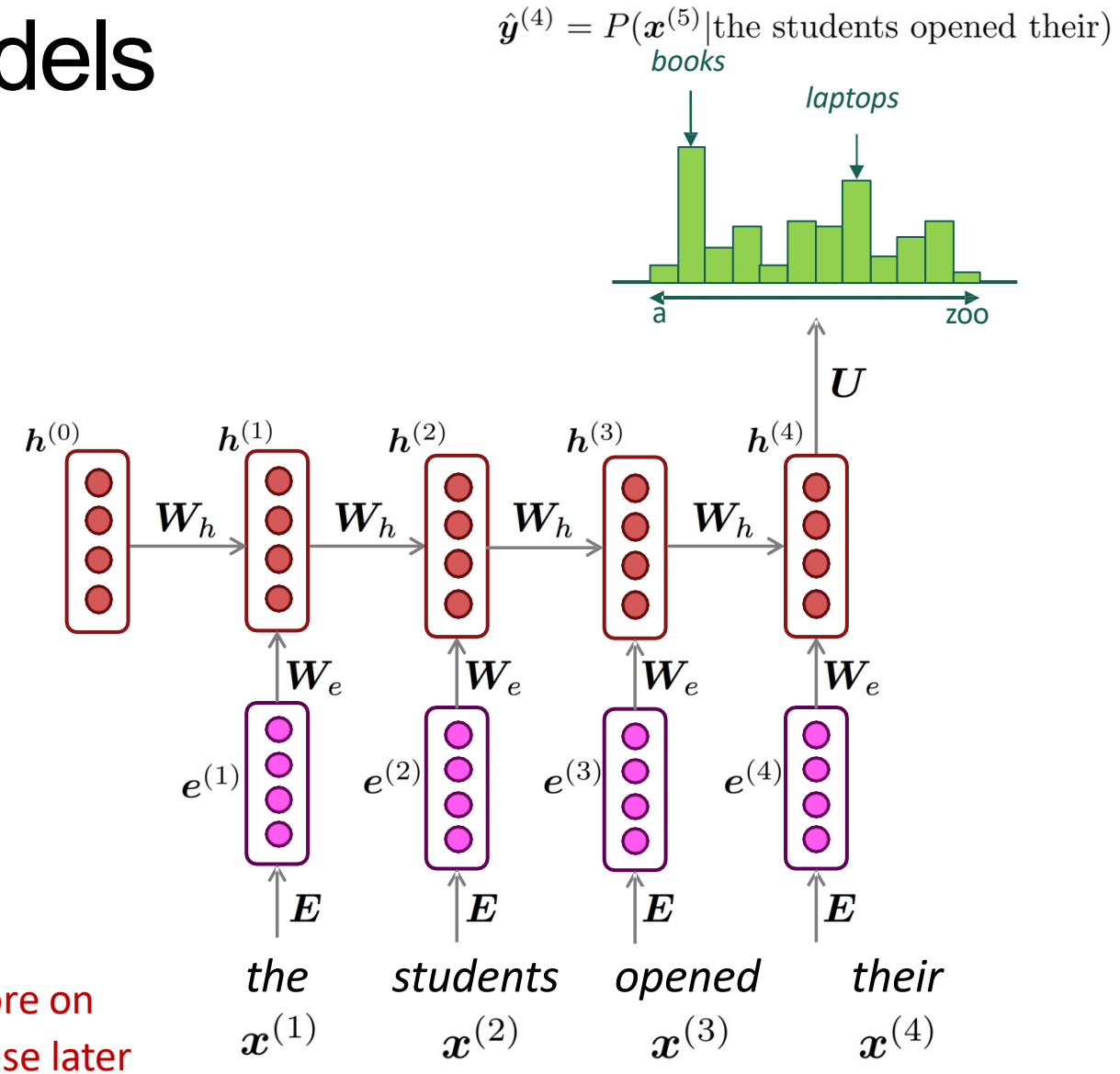
RNN **Advantages**:

- Can process any length input
- Computation for step *t* can (in theory) use information from many steps back
- Model size doesn't increase for longer input context
- Same weights applied on every timestep, so there is symmetry in how inputs are processed.

RNN **Disadvantages**:

- Recurrent computation is slow
- In practice, difficult to access info from many steps back

More on these later



Exercise

You are building an **RNN-based language model** to predict the next word in a sentence.

- Vocabulary size = **5 words**: {I, he, go, walk, eat}
- Each word is encoded as a **one-hot vector of size 5**
- Hidden layer size = **6 units**
- Bias terms are **ignored**

Question:

How many **trainable parameters (weights)** does this RNN have?

Exercise

1. Input → Hidden weights (Wxh)

- Input size = 5 (one-hot vector)
- Hidden size = 6

So matrix size: 5×6=30

2. Hidden → Hidden weights (Whh)

- Hidden size = 6
- Hidden size = 6

So matrix size: 6×6=36

3. Hidden → Output weights (Why)

- Hidden size = 6
- Output size = 5 (vocabulary size)

So matrix size: 6×5=30

Total Number of Parameters

Add all three: 30+36+30=96

Training an RNN Language Model

- Get a big corpus of text which is a sequence of words $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{\boldsymbol{y}}^{(t)}$ for *every step t*.
  - i.e., predict probability dist of *every word*, given words so far

- Loss function on step *t* is cross-entropy between predicted probability distribution $\hat{\boldsymbol{y}}^{(t)}$, and the true next word $\boldsymbol{y}^{(t)}$ (one-hot for $\boldsymbol{x}^{(t+1)}$):

$$J^{(t)}(\theta) = CE(\boldsymbol{y}^{(t)}, \hat{\boldsymbol{y}}^{(t)}) = -\sum_{w \in V} \boldsymbol{y}_w^{(t)} \log \hat{\boldsymbol{y}}_w^{(t)} = -\log \hat{\boldsymbol{y}}_{\boldsymbol{x}_{t+1}}^{(t)}$$

- Average this to get overall loss for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^{T} -\log \hat{\boldsymbol{y}}_{\boldsymbol{x}_{t+1}}^{(t)}$$

Cross Entropy Loss

$$J^{(t)}(\theta) = CE(\boldsymbol{y}^{(t)}, \hat{\boldsymbol{y}}^{(t)}) = -\sum_{w \in V} \boldsymbol{y}_w^{(t)} \log \hat{\boldsymbol{y}}_w^{(t)} = -\log \hat{\boldsymbol{y}}_{\boldsymbol{x}_{t+1}}^{(t)}$$

What is $x_{(t+1)}$?

It is the **actual next word** in the training data.

Gold: [0, 1, 0, 0]
Predicted: [0.1, 0.6, 0.2, 0.1]

$$= -(0 \log 0.1 + 1 \log 0.6 + 0 \log 0.2 + 0 \log 0.1)$$

$$= -\log(0.6)$$

Loss≈0.51

Predicted: [0.4, 0.3, 0.05, 0.25]

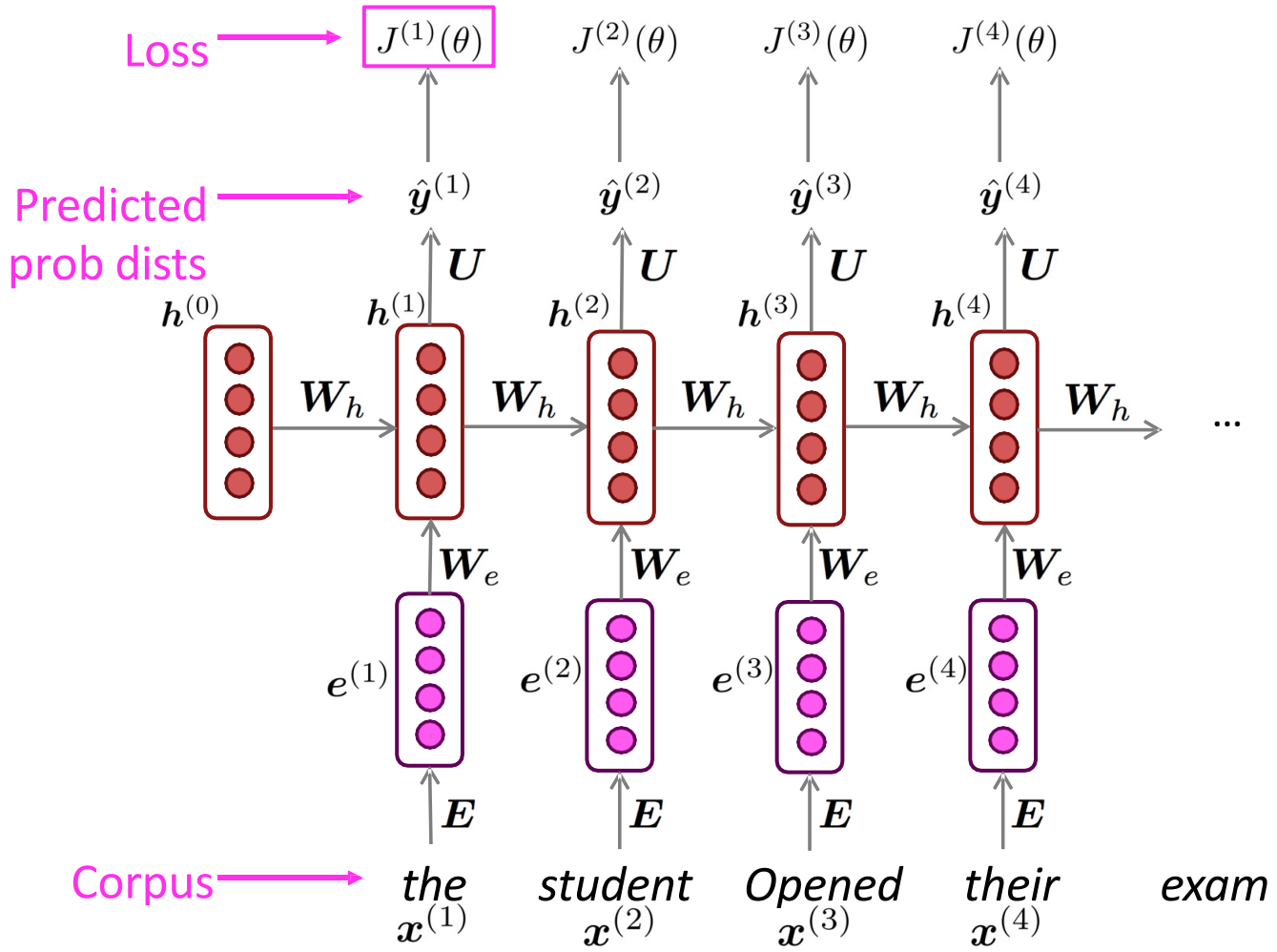
$$= -\log(0.05)$$
$$= 2.99$$

- ✔ Converts multiplication into addition
- ✔ Strong penalty for confident wrong predictions
- ✔ Stable gradients
- ✔ Works well with softmax

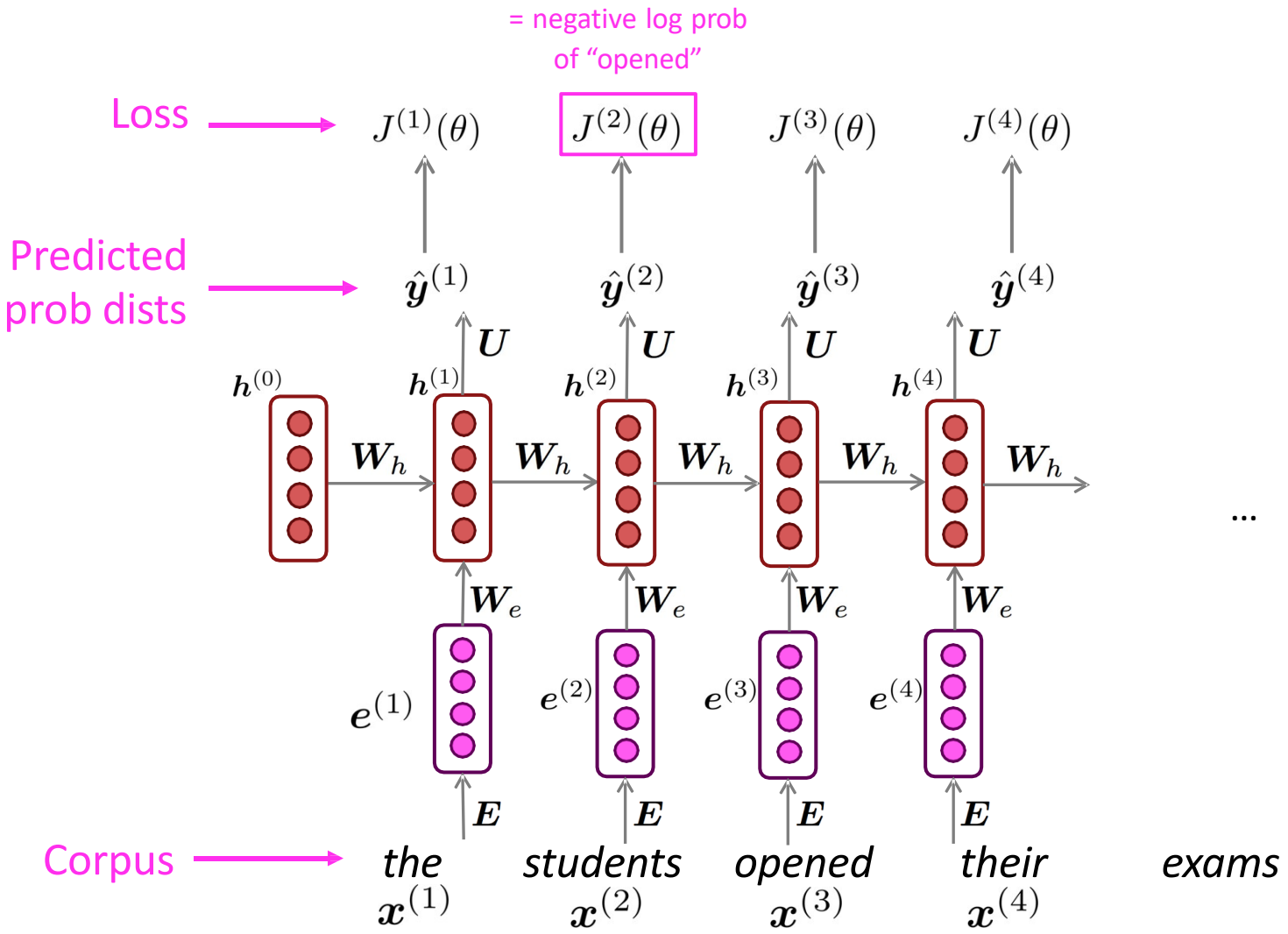
BCE $= -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$

Training an RNN Language Model

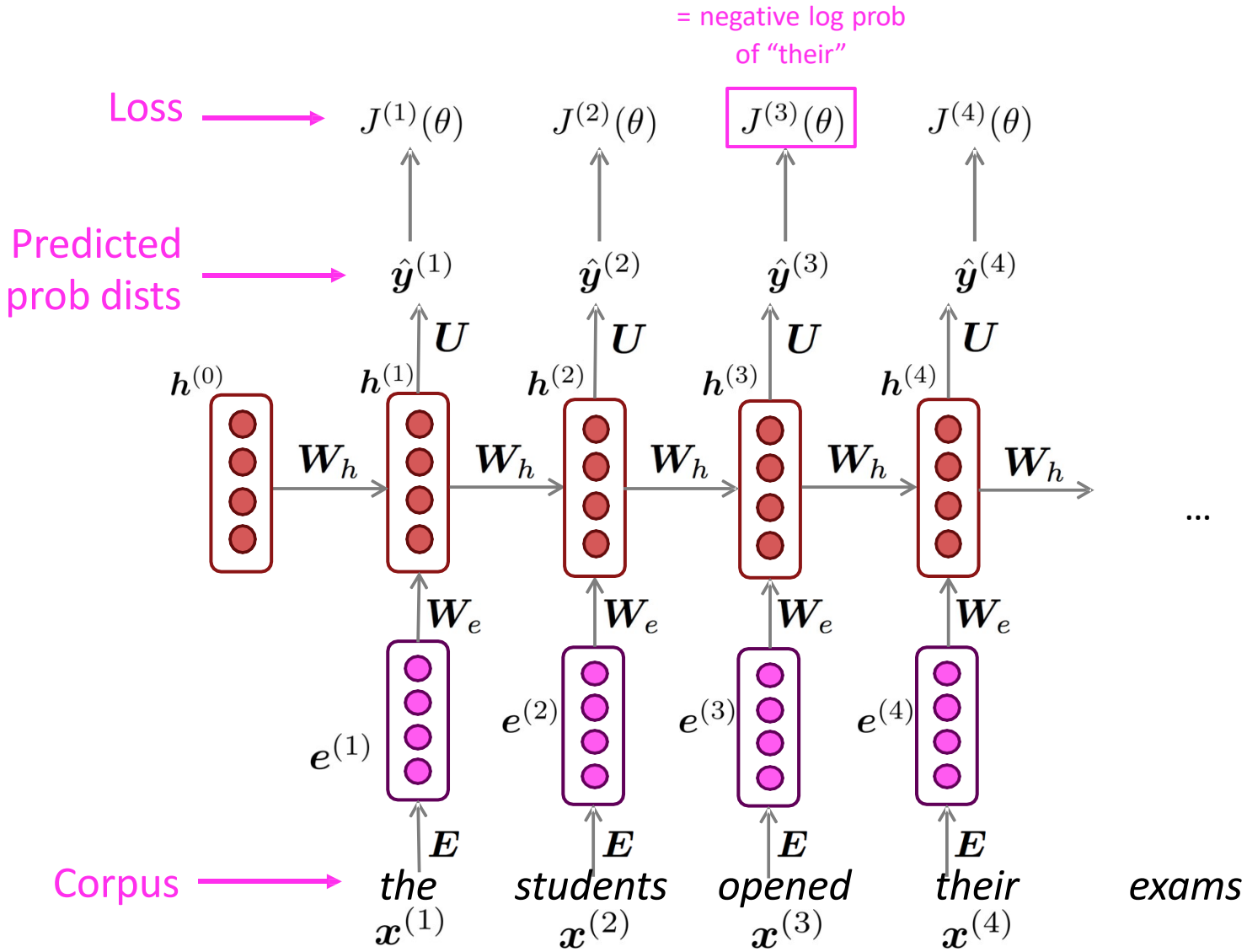
= negative log prob of "students"



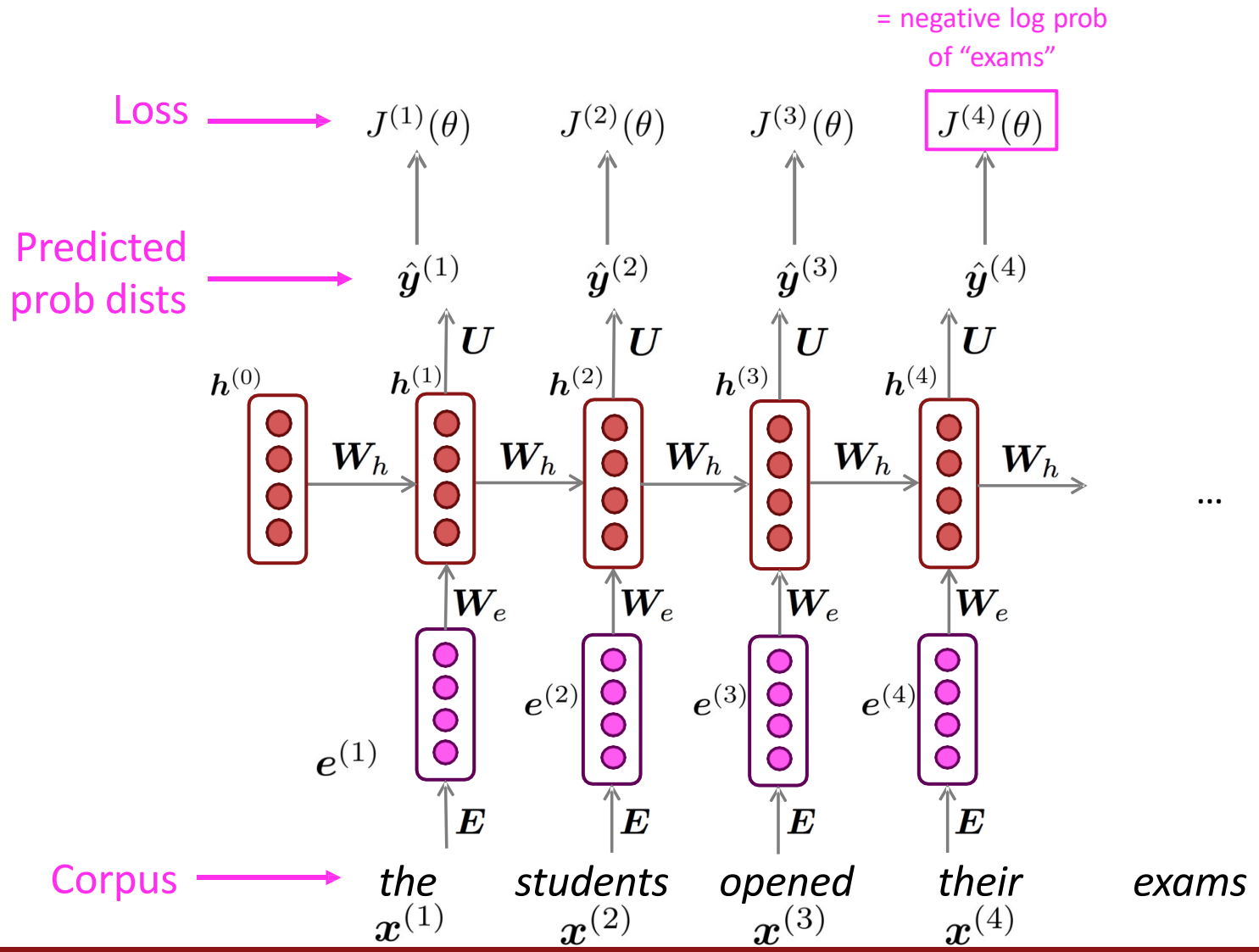
Training an RNN Language Model



Training an RNN Language Model

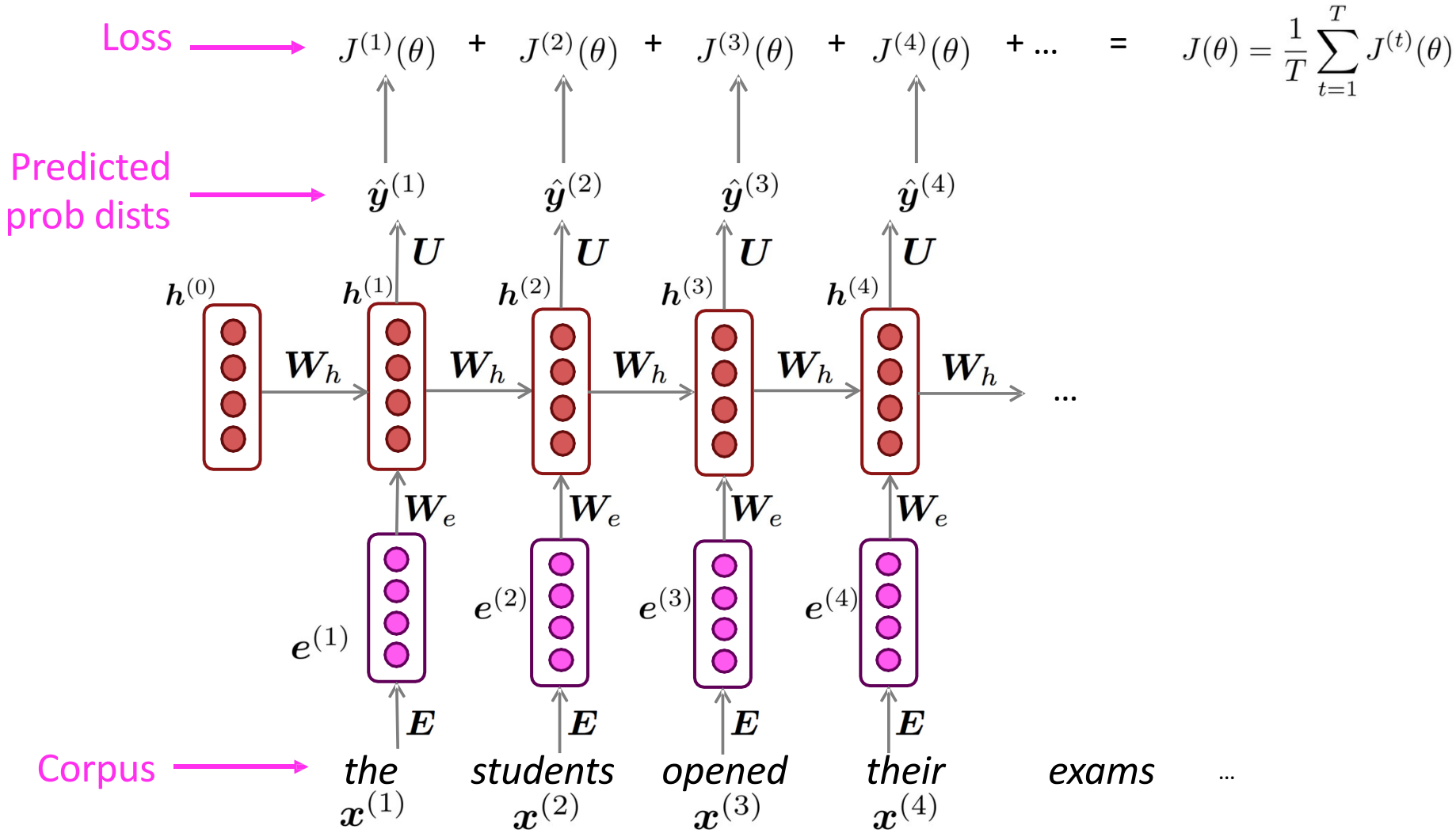


Training an RNN Language Model



Training an RNN Language Model

"Teacher forcing"



Training a RNN Language Model

- However: Computing loss and gradients across entire corpus $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$ at once is too expensive (memory-wise)!

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J^{(t)}(\theta)$$

- In practice, consider $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}$ as a sentence (or a document)

- Recall: Stochastic Gradient Descent allows us to compute loss and gradients for small chunk of data, and update.

- Compute loss $J(\theta)$ for a sentence (actually, a batch of sentences), compute gradients and update weights. Repeat on a new batch of sentences.

- RNNs for Text Generation
- RNNs for Sequence Labeling
- RNNs for Text Classification

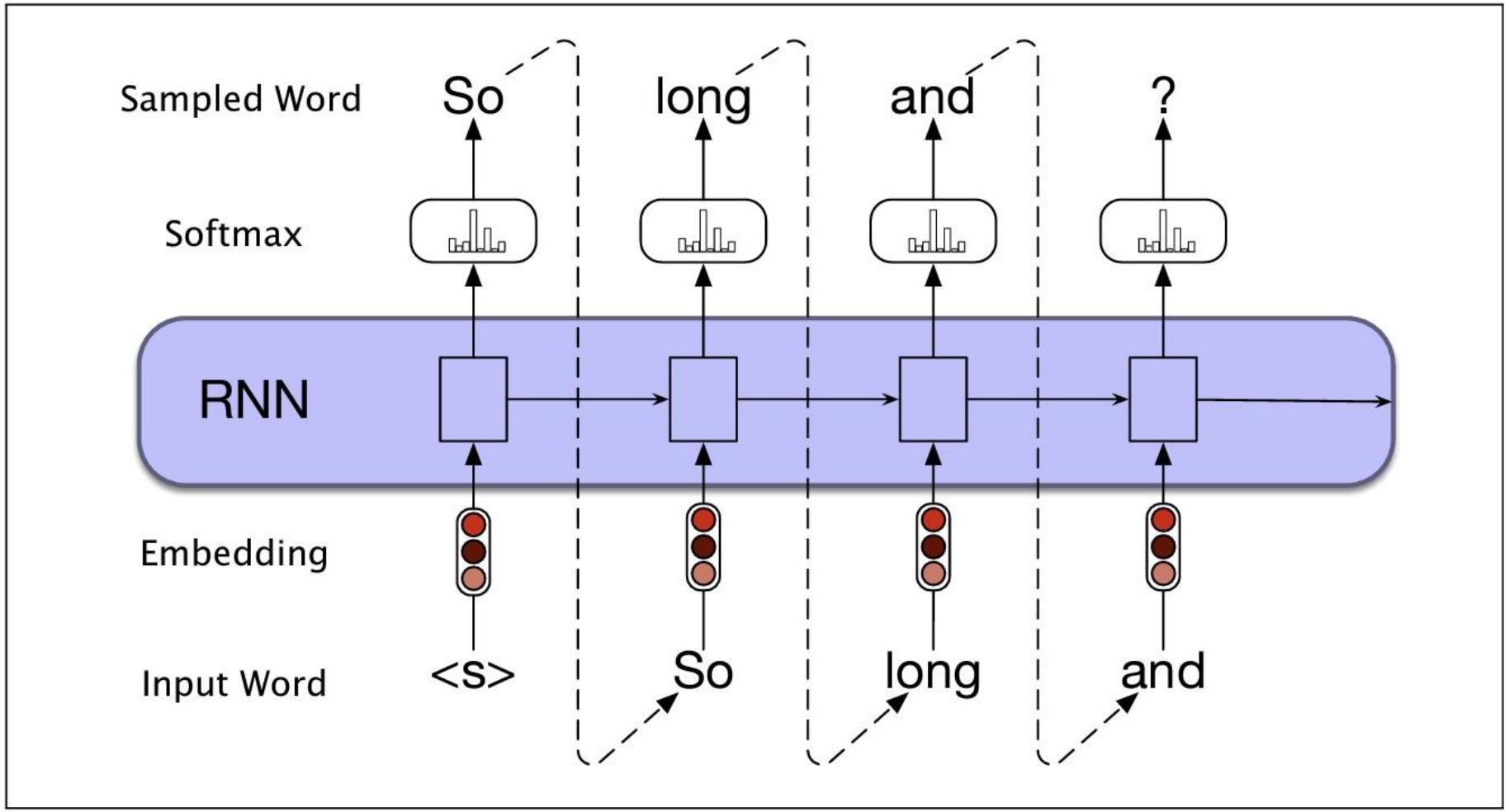
Generating text with an RNN Language Model

- RNN-based language models can be used for language generation (and hence, for machine translation, dialog, etc.)
- A language model can incrementally generate words by repeatedly sampling the words conditioned on the previous choices – also known as *autoregressive generation*.

Autoregressive Generation with RNNs

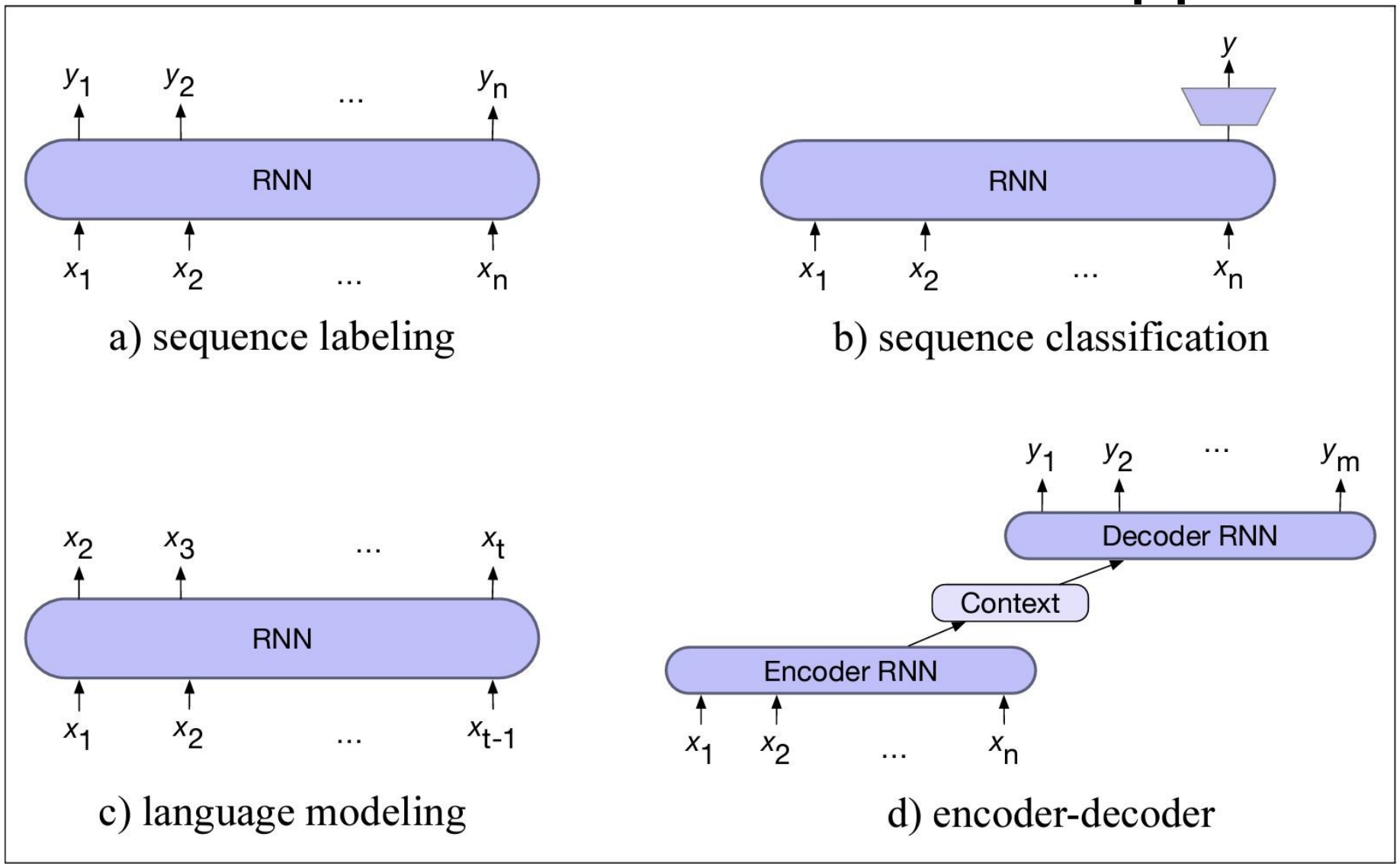
- All your parameters have already been trained.
- Start with a special begin of sentence token <s> as input
- Through forward propagation, obtain the probability distribution at the output, and sample a word
- Feed the word as input at the next time-step (its word vector)
- Continue generating until the end of sentence token is sampled, or a fixed length of the sentence has been reached.

Autoregressive Generation with RNNs



Source: https://web.stanford.edu/~jurafsky/slp3.

RNNs can be used for various applications



Source: https://web.stanford.edu/~jurafsky/slp3.

RNNs for Sequence Labeling

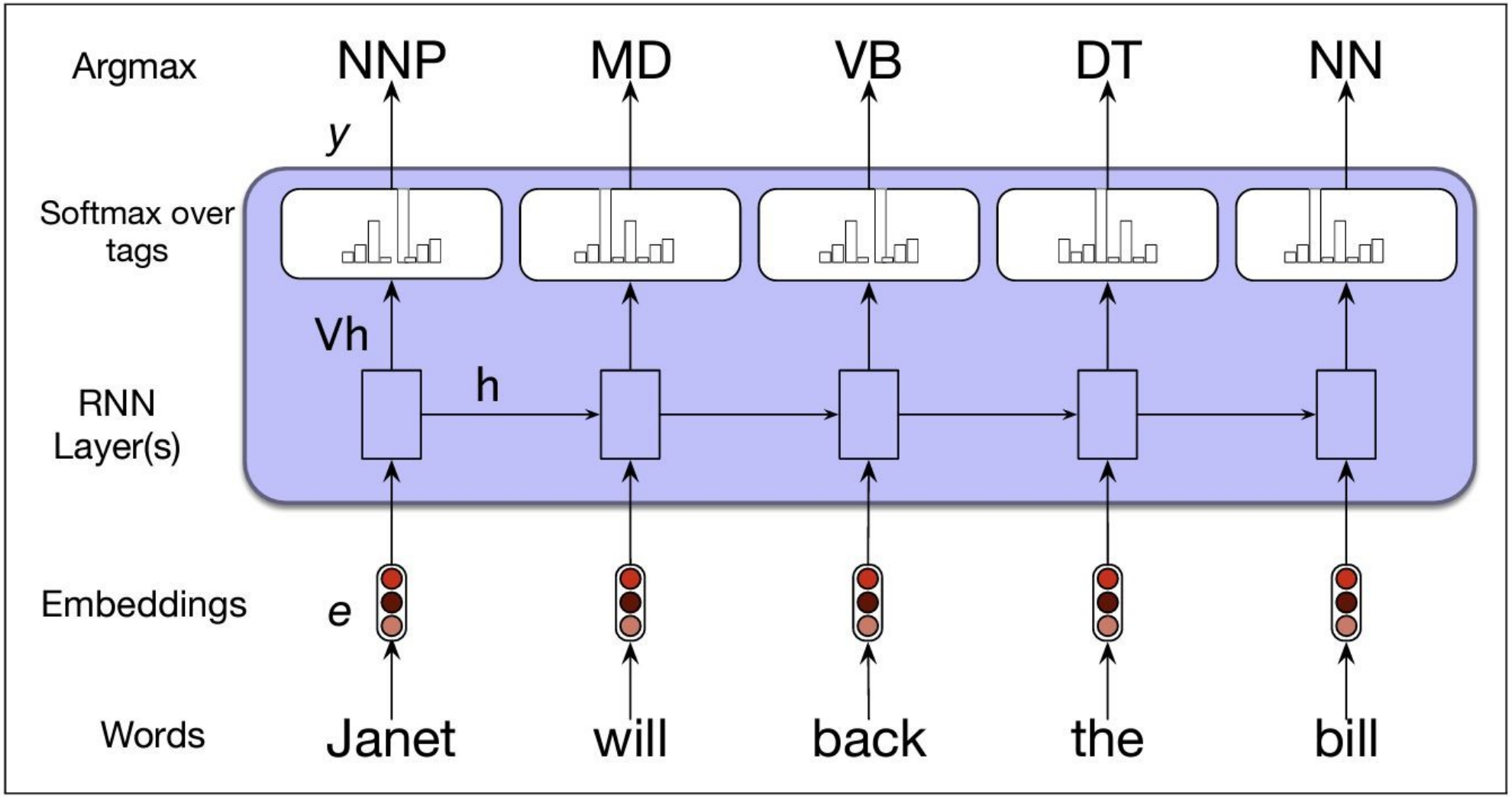
Task

Assign a label chosen from a small fixed set of labels to each element of the sequence

- Inputs: Word embeddings
- Outputs: Tag probabilities generated by the softmax layer

Source: https://web.stanford.edu/~jurafsky/slp3.

RNNs for Sequence Labeling: POS Tagging



Source: https://web.stanford.edu/~jurafsky/slp3.

Named Entities

- **Named entity**, in its core usage, means anything that can be referred to with a proper name. Most common 4 tags:
  - **PER (Person): "Marie Curie"**
  - **LOC (Location): "New York City"**
  - **ORG (Organization): "Stanford University"**
  - **GPE (Geo-Political Entity): "Boulder, Colorado"**
- **Often multi-word phrases**
- **But the term is also extended to things that aren't entities:**
  - **dates, times, prices**

Source: https://web.stanford.edu/~jurafsky/slp3.

Named Entity tagging

The task of named entity recognition (NER):

- find spans of text that constitute proper names
- tag the type of the entity.

Source: https://web.stanford.edu/~jurafsky/slp3.

NER output

Citing high fuel prices, [ORG **United Airlines**] said [TIME **Friday**] it has increased fares by [MONEY **$6**] per round trip on flights to some cities also served by lower-cost carriers. [ORG **American Airlines**], a unit of [ORG **AMR Corp.**], immediately matched the move, spokesman [PER **Tim Wagner**] said. [ORG **United**], a unit of [ORG **UAL Corp.**], said the increase took effect [TIME **Thursday**] and applies to most routes where it competes against discount carriers, such as [LOC **Chicago**] to [LOC **Dallas**] and [LOC **Denver**] to [LOC **San Francisco**].

Source: https://web.stanford.edu/~jurafsky/slp3.

Why NER?

Sentiment analysis: consumer's sentiment toward a particular company or person?

Question Answering: answer questions about an entity?

Information Extraction: Extracting facts about entities from text.

Why NER is hard

1) Segmentation

- In POS tagging, no segmentation problem since each word gets one tag.
- In NER we have to find and segment the entities!

2) Type ambiguity

[PER Washington] was born into slavery on the farm of James Burroughs.
[ORG Washington] went up 2 games to 1 in the four-game series.
Blair arrived in [LOC Washington] for what may well be his last state visit.
In June, [GPE Washington] passed a primary seatbelt law.

Source: https://web.stanford.edu/~jurafsky/slp3.

BIO Tagging

How can we turn this structured problem into a sequence problem like POS tagging, with one label per word?

[PER Jane Villanueva] of [ORG United] , a unit of [ORG United Airlines Holding] , said the fare applies to the [LOC Chicago] route.

Source: https://web.stanford.edu/~jurafsky/slp3.

BIO Tagging

[PER Jane Villanueva] of [ORG United] , a unit of [ORG United Airlines Holding] , said the fare applies to the [LOC Chicago] route.

Words	BIO Label
Jane	B-PER
Villanueva	I-PER
of	O
United	B-ORG
Airlines	I-ORG
Holding	I-ORG
discussed	O
the	O
Chicago	B-LOC
route	O
.	O

Now we have one tag per token!!!

Source: https://web.stanford.edu/~jurafsky/slp3.

BIO Tagging

B: token that *begins* a span

I: tokens *inside* a span

O: tokens outside of any span

of tags (where n is #entity types):

1 O tag,

n B tags,

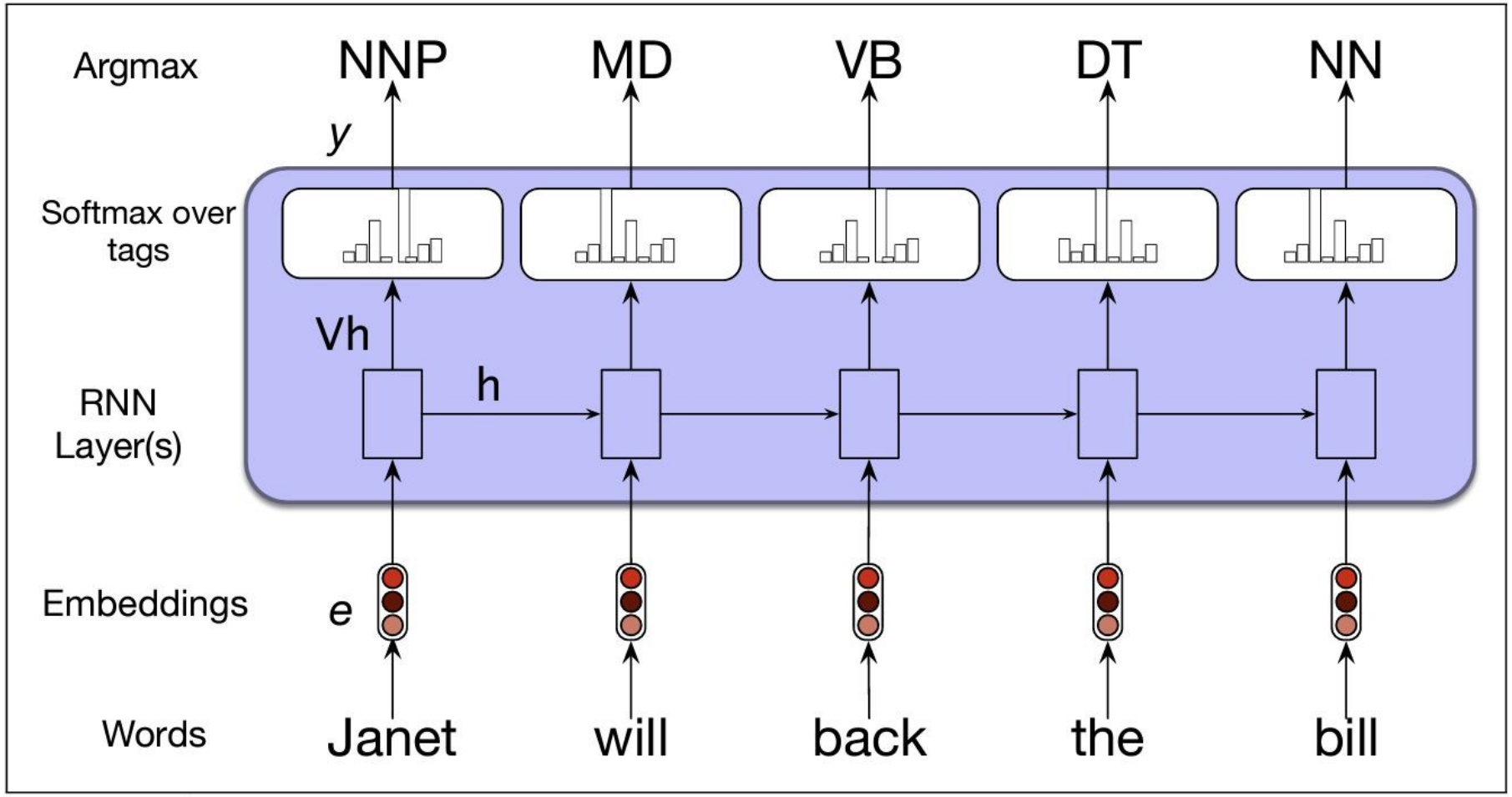
n I tags

total of *2n+1*

Words	BIO Label
Jane	B-PER
Villanueva	I-PER
of	O
United	B-ORG
Airlines	I-ORG
Holding	I-ORG
discussed	O
the	O
Chicago	B-LOC
route	O
.	O

Source: https://web.stanford.edu/~jurafsky/slp3.

RNNs for Sequence Labeling



Suppose you are using 300-dim word embeddings, 50-dim hidden vector, and you have 50 POS tags. What will be the number of parameters excluding the bias terms?

RNNs for Sequence Labeling

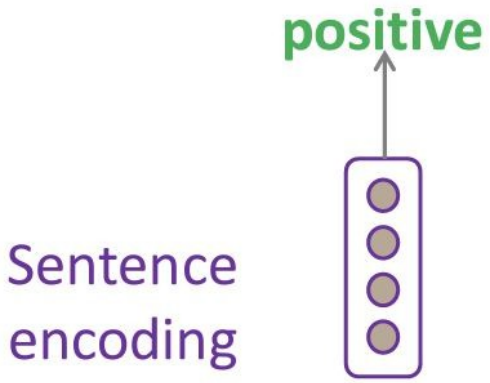
Suppose you are using 300-dim word embeddings, 50-dim hidden vector, and you have ***5 NER tags, and you are using BIO tagging scheme***. What will be the number of parameters excluding the bias terms?

BIO tagging creates:

- **B-** tag for each entity
- **I-** tag for each entity
- Plus **O** (Outside) tag

So total tags: $=2\times5+1=11$ (output labels)

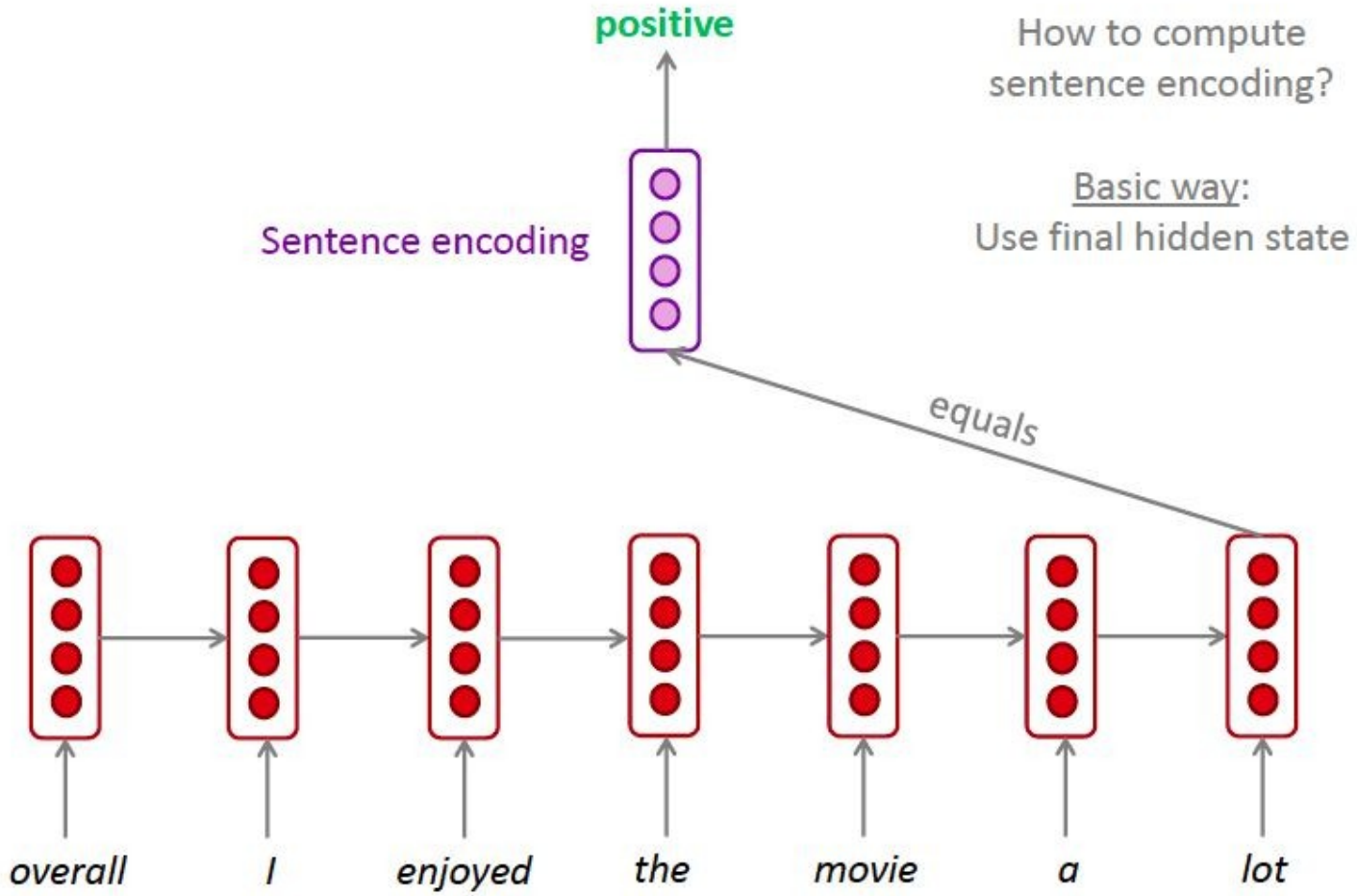
How to use RNNs for Text classification?



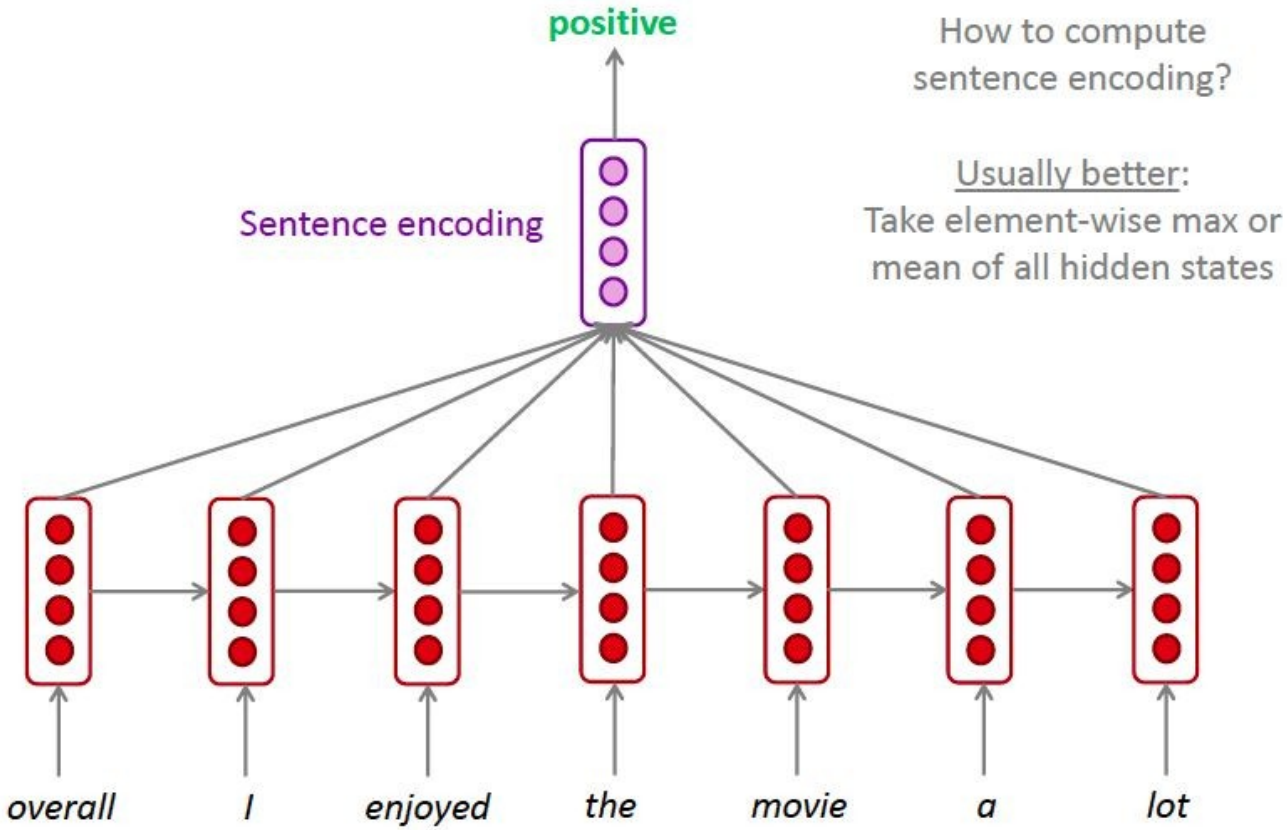
How to compute sentence encoding?

overall *I* *enjoyed* *the* *movie* *a* *lot*

RNNs for Sentence Classification

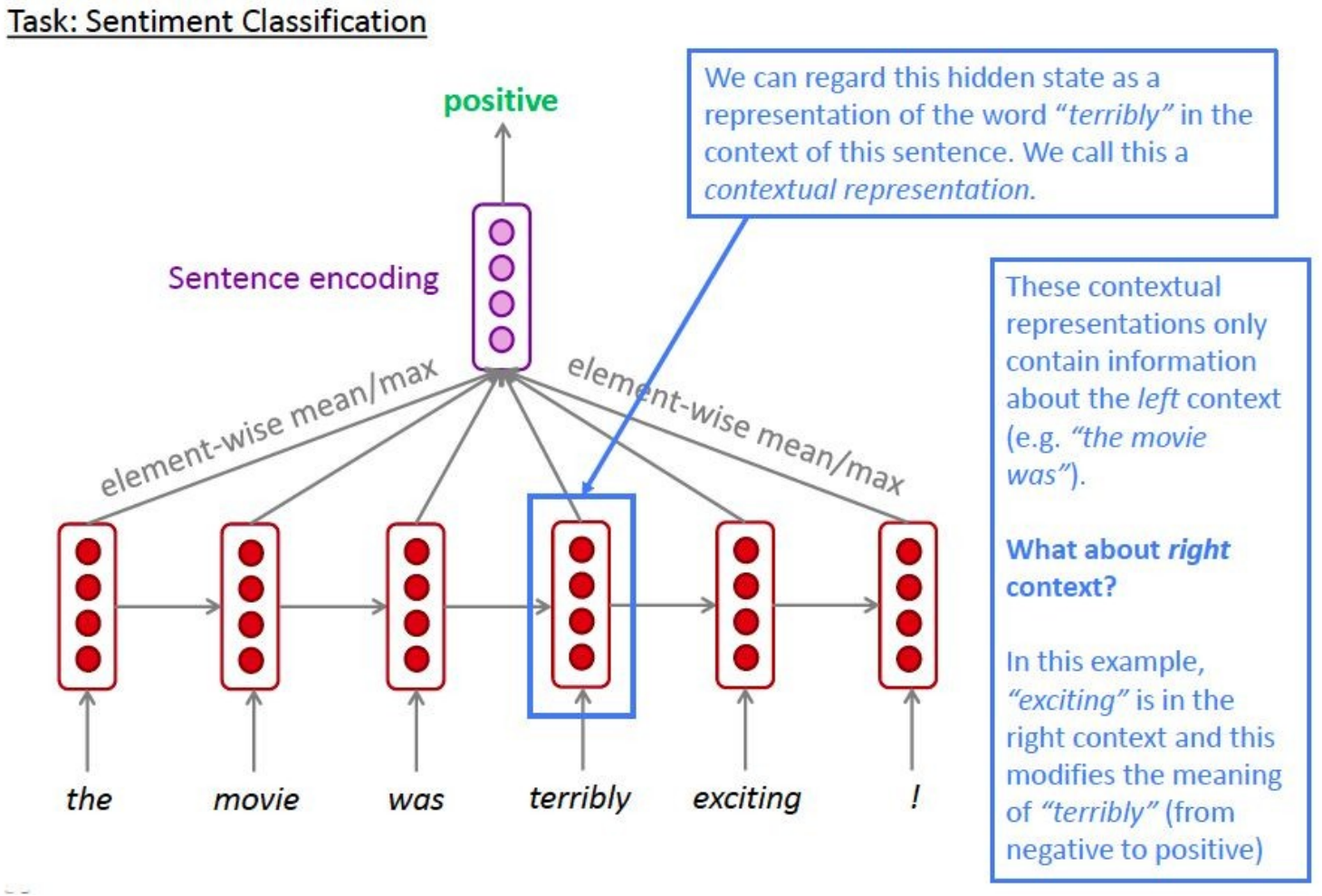


RNNs for Sentence Classification



Source: https://web.stanford.edu/class/cs224n/

Bidirectional RNNs: motivation for sentiment classification



Bidirectional RNNs: Formally

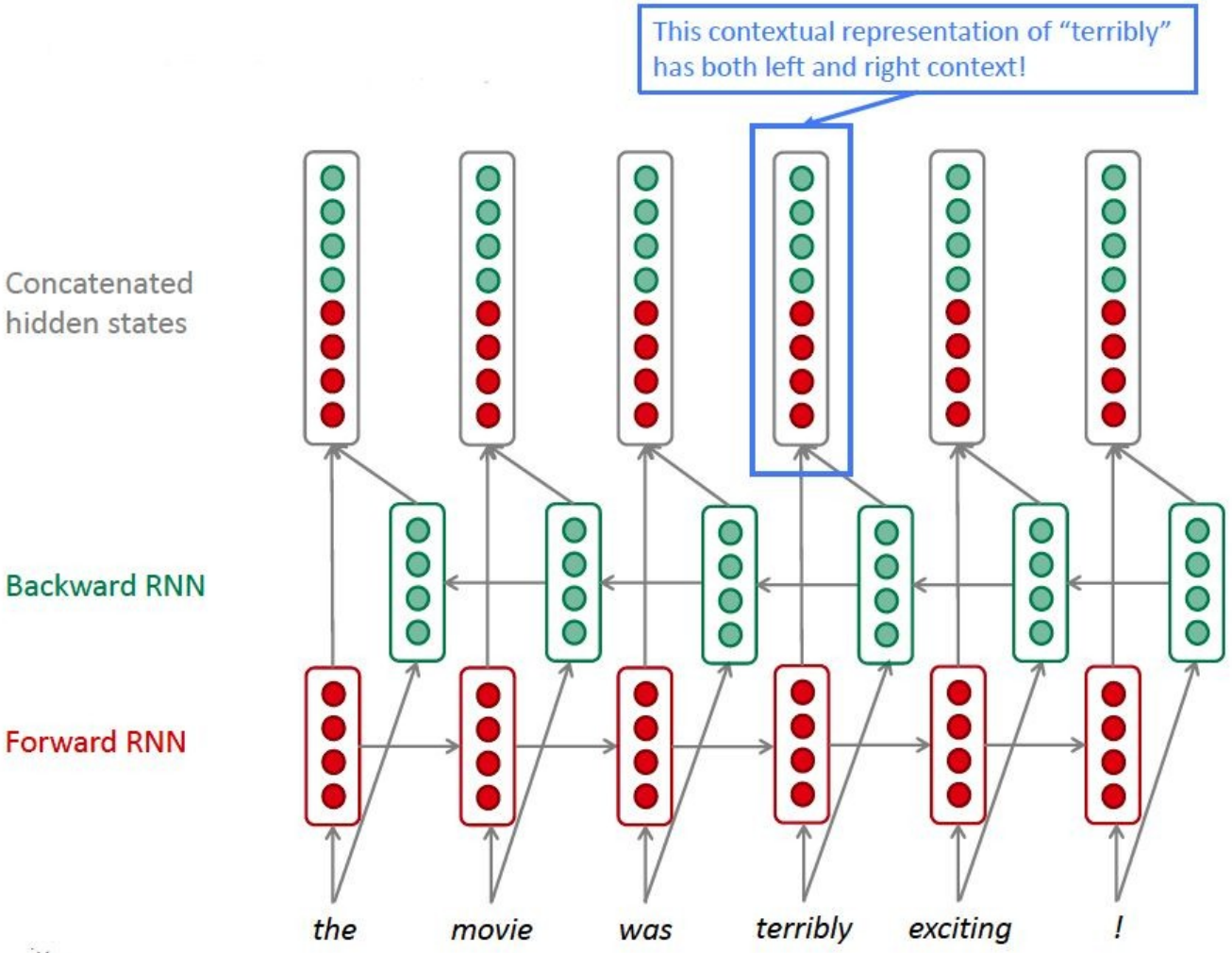
- RNN makes use of information from left (prior) context to predict at time t
- In many applications, the entire sequence is available; so it makes sense to also make use of the right context to predict at time t
- Bidirectional RNNs combine two independent RNNs, one where the input is processed from left to right (forward RNN), and another from end to the start (backward RNN).

$$h_t^f = RNN_{forward}(x_1, \ldots, x_t)$$

$$h_t^b = RNN_{backward}(x_n, \ldots, x_t)$$

$$h_t = [h_t^f; h_t^b]$$

Bidirectional RNNs



Source: https://web.stanford.edu/class/cs224n/

Evaluating Language Models

- The standard evaluation metric for Language Models is perplexity.

$$\text{perplexity} = \prod_{t=1}^{T} \left(\frac{1}{P_{\text{LM}}(\boldsymbol{x}^{(t+1)} | \ \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)})} \right)^{1/T}$$

Normalized by number of words

Inverse probability of corpus, according to Language Model

- This is equal to the exponential of the cross-entropy loss $J(\theta)$:

$$= \prod_{t=1}^{T} \left(\frac{1}{\hat{\boldsymbol{y}}^{(t)}_{\boldsymbol{x}_{t+1}}} \right)^{1/T} = \exp\left(\frac{1}{T} \sum_{t=1}^{T} -\log \hat{\boldsymbol{y}}^{(t)}_{\boldsymbol{x}_{t+1}} \right) = \exp(J(\theta))$$

Lower perplexity is better!

Interpretation

Perplexity = 1.44 means:
On average, the model is choosing between about **1.44 possible words** at each prediction.

Perplexity = 4.07 means:

Model confused
Almost guessing among 4 words

If the model assigns high probability to correct words, loss is low and perplexity is low. If the model assigns low probability to correct words, loss and perplexity become high.

Vanishing Gradients

What is it?

- During backpropagation, gradients are propagated from the output layer towards earlier layers.
- If gradients become extremely small, the earlier layers learn very slowly or stop learning.

Why does it happen? Using the chain rule, the gradient across multiple layers is a product of derivatives:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_n} \prod_{l=1}^{n-1} \frac{\partial h_{l+1}}{\partial h_l}$$

If the derivatives have magnitudes smaller than 1, their product can approach zero. **Example:** If each layer contributes a derivative of 0.5, then

$$(0.5)^{10} \approx 0.00098$$

After many layers, the gradient becomes very small.

Consequences
Early layers learn very slowly.
Training deep networks becomes difficult.

Solutions: ReLU activations, suitable weight initialization, residual connections, and LSTM/GRU for sequential data

Exploding Gradients

What is it?

- During backpropagation, gradients can grow extremely large as they pass through multiple layers.
- This can cause excessively large weight updates.

Why does it happen?

If the derivatives propagated across layers have magnitudes greater than 1, their product can grow exponentially.

Example: If each layer contributes a derivative of 2, then

$$2^{10} = 1024$$

A small gradient at the output can become very large in earlier layers.

Consequences

- Very large and unstable weight updates.
- Loss may fluctuate, become NaN, or diverge.

Solutions:

- **Gradient clipping:** Limit the gradient norm.

$$g_{\text{clipped}} = g \min\left(1, \frac{\tau}{\|g\|}\right)$$

  where τ is the maximum allowed gradient norm.
- Use suitable initialization and learning rates.
- Use residual connections and appropriate architectures.

LSTM and GRU

For example, in an **LSTM**:

$$
\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t]) && \text{forget gate (0–1)} \\
i_t &= \sigma(W_i[h_{t-1}, x_t]) && \text{input gate (0–1)} \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{cell memory} \\
h_t &= o_t \odot \tanh(c_t) && \text{output gate}
\end{aligned}
$$

Here:

- f_t controls **when to forget** old info.
- i_t controls **when to remember** new info.
- o_t controls **what to output**.

- Solution 2: And then: Creating more direct and linear pass-through connections in model
  - Attention, residual connections, etc.

Machine Translation

Machine Translation (MT) is the task of translating a sentence *x* from one language (the source language) to a sentence *y* in another language (the target language).

x: *I like deep learning*

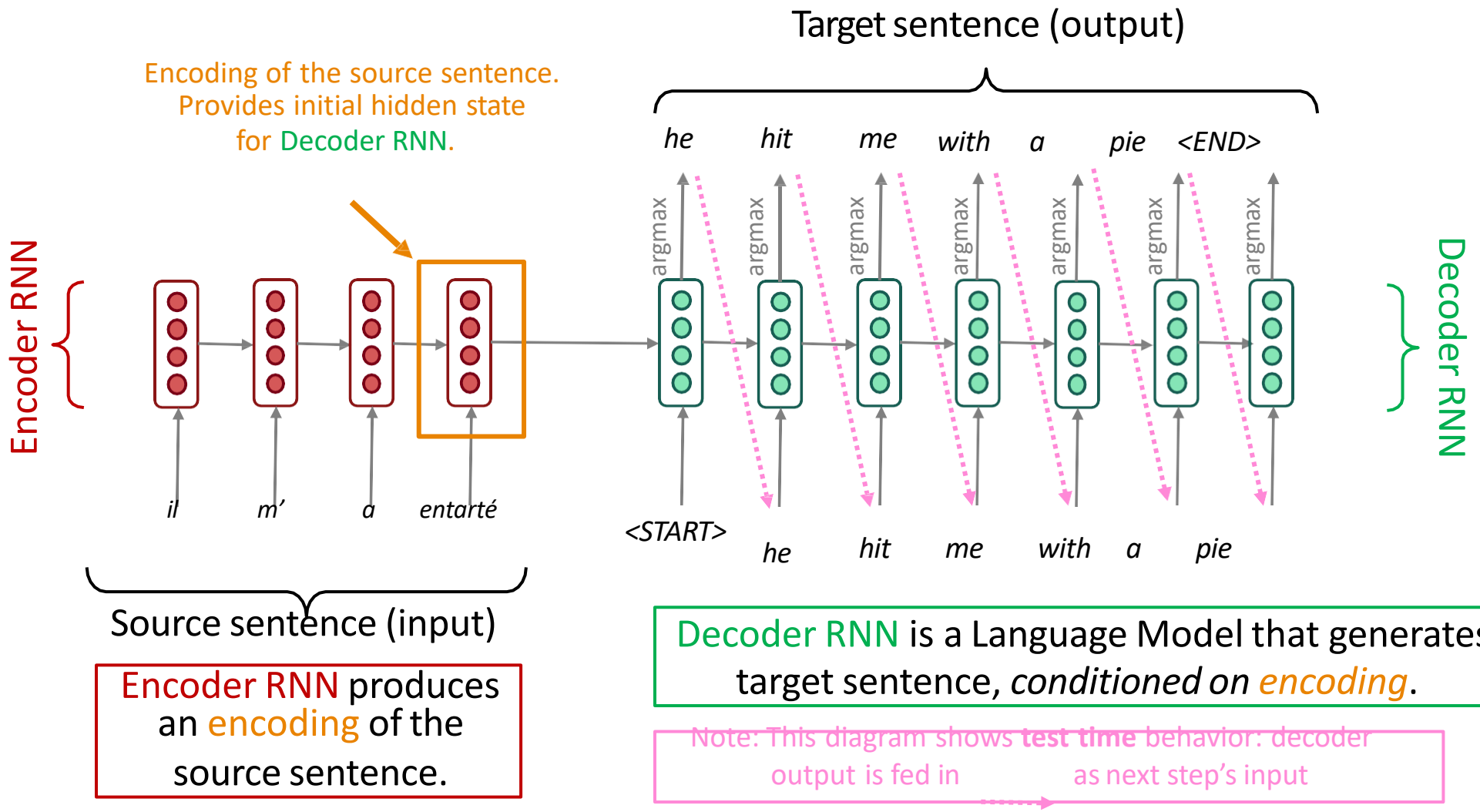


y: 我喜欢深度学习

Implementation: https://github.com/bentrevett/pytorch-seq2seq

Neural Machine Translation (NMT)

The sequence-to-sequence model



$$h_t = g_\theta(h_{t-1}, x_t)$$
$$\downarrow$$
$$h_t = \tanh(W_{hh}\, h_{t-1} + W_{xh} x_t)$$

$$o_t = softmax(W_{ho} h_t)$$

Encoder

For input sequence:

$$x_1, x_2, \ldots, x_T$$

Hidden state update:

$$h_t = \tanh(W_e x_t + U_e h_{t-1})$$

Final context vector:

$$c = h_T$$

Decoder

Initialize decoder state:

$$s_0 = c$$

Update rule:

$$s_t = \tanh(W_d y_{t-1} + U_d s_{t-1})$$

Output:

$$y_t = \text{softmax}(V s_t)$$

Sequence-to-sequence is versatile!

- The general notion here is an encoder-decoder model
  - One neural network takes input and produces a neural representation
  - Another network produces output based on that neural representation
  - If the input and output are sequences, we call it a seq2seq model

- Sequence-to-sequence is useful for *more than just MT*
- Many NLP tasks can be phrased as sequence-to-sequence:
  - Summarization (long text → short text)
  - Dialogue (previous utterances → next utterance)
  - Parsing (input text → output parse as sequence)
  - Code generation (natural language → Python code)

Neural Machine Translation (NMT)

- The sequence-to-sequence model is an example of a **Conditional Language Model**
  - **Language Model** because the decoder is predicting the next word of the target sentence *y*
  - **Conditional** because its predictions are *also* conditioned on the source sentence *x*

- NMT directly calculates $P(y|x)$

$$P(y|x) = P(y_1|x)\, P(y_2|y_1, x)\, P(y_3|y_1, y_2, x) \ldots \underbrace{P(y_T|y_1, \ldots, y_{T-1}, x)}$$

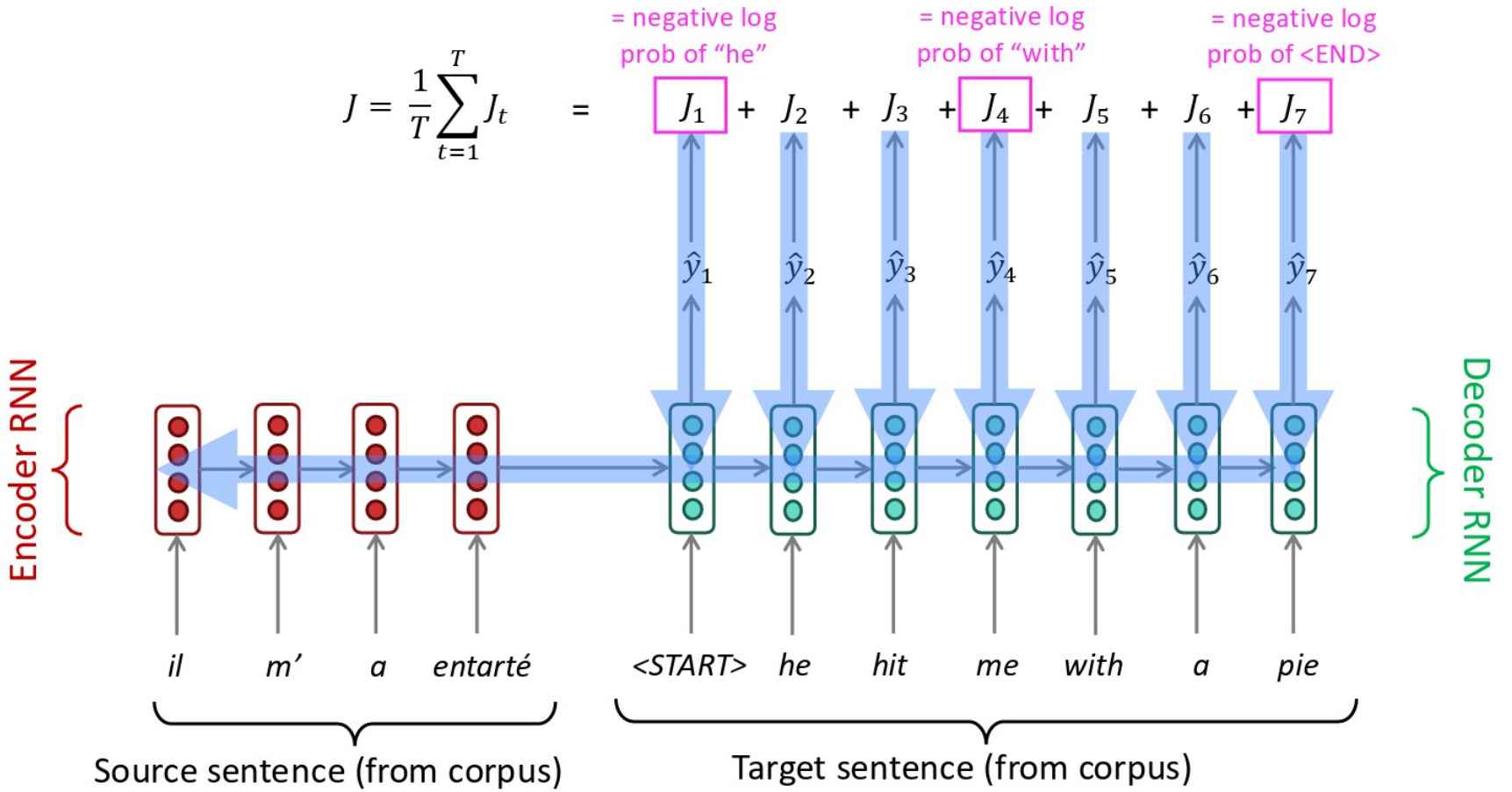
Probability of next target word, given target words so far and source sentence *x*

It conditions on:
- previous target words
- **entire source sentence x**

This extra conditioning is the key difference.

- **Question:** How to train an NMT system?
- **(Easy) Answer:** Get a big parallel corpus…
  - But there is now exciting work on "unsupervised NMT", data augmentation, etc.

Training a Neural Machine Translation system

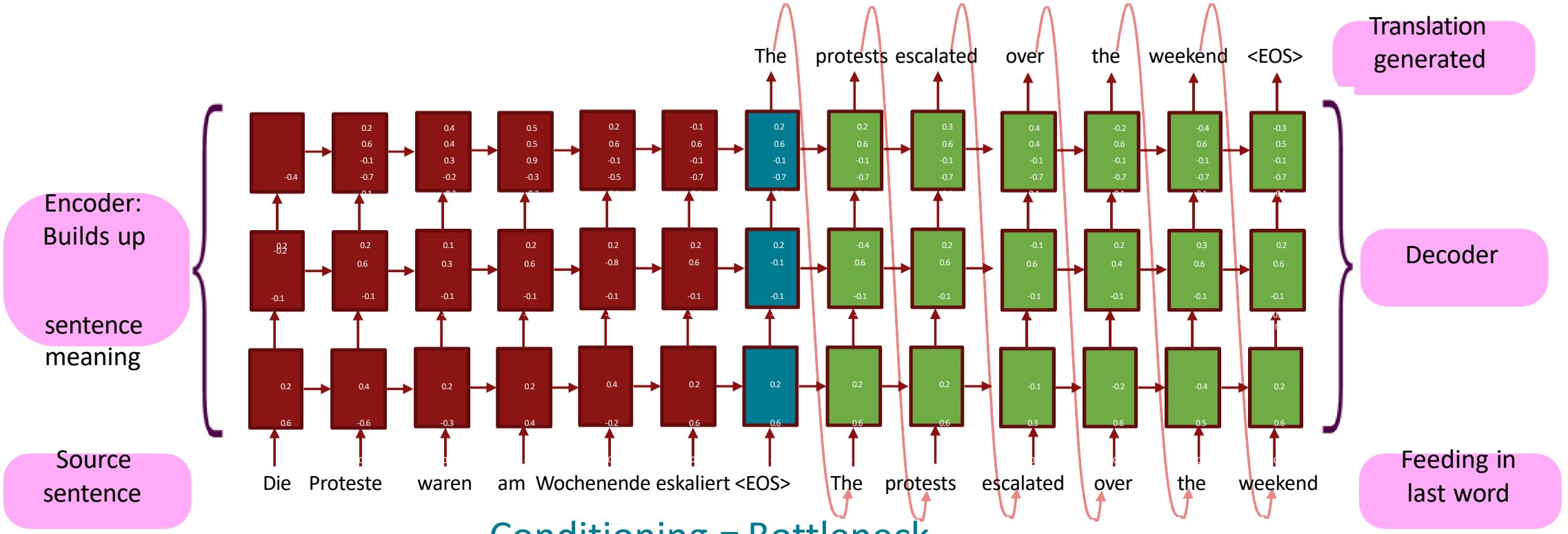


Seq2seq is optimized as a **single system.** Backpropagation operates *"end-to-end"*.

Multi-layer deep encoder-decoder machine translation net

[Sutskever et al. 2014; Luong et al. 2015]

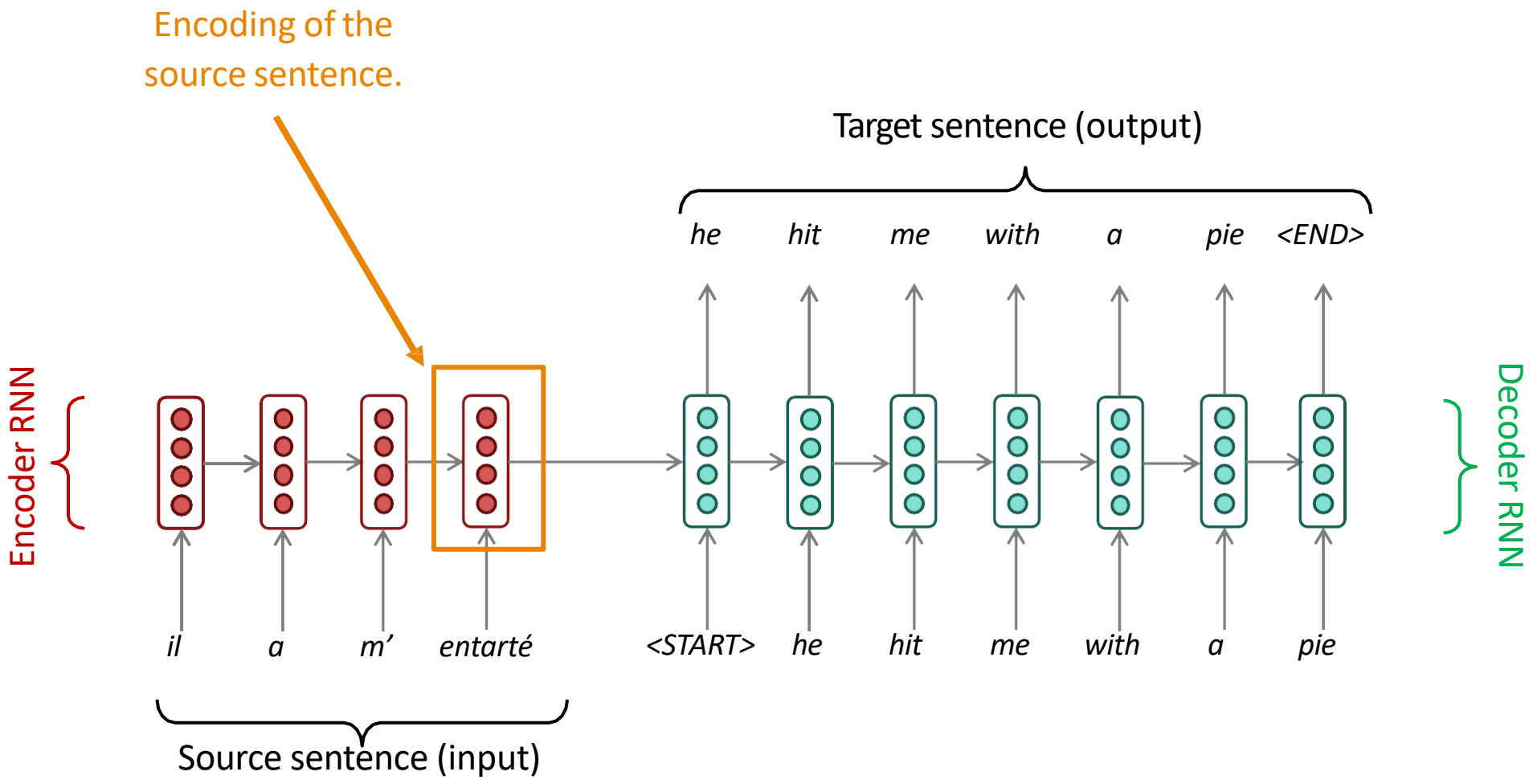
The hidden states from RNN layer *i* are the inputs to RNN layer *i*+1



Conditioning = Bottleneck

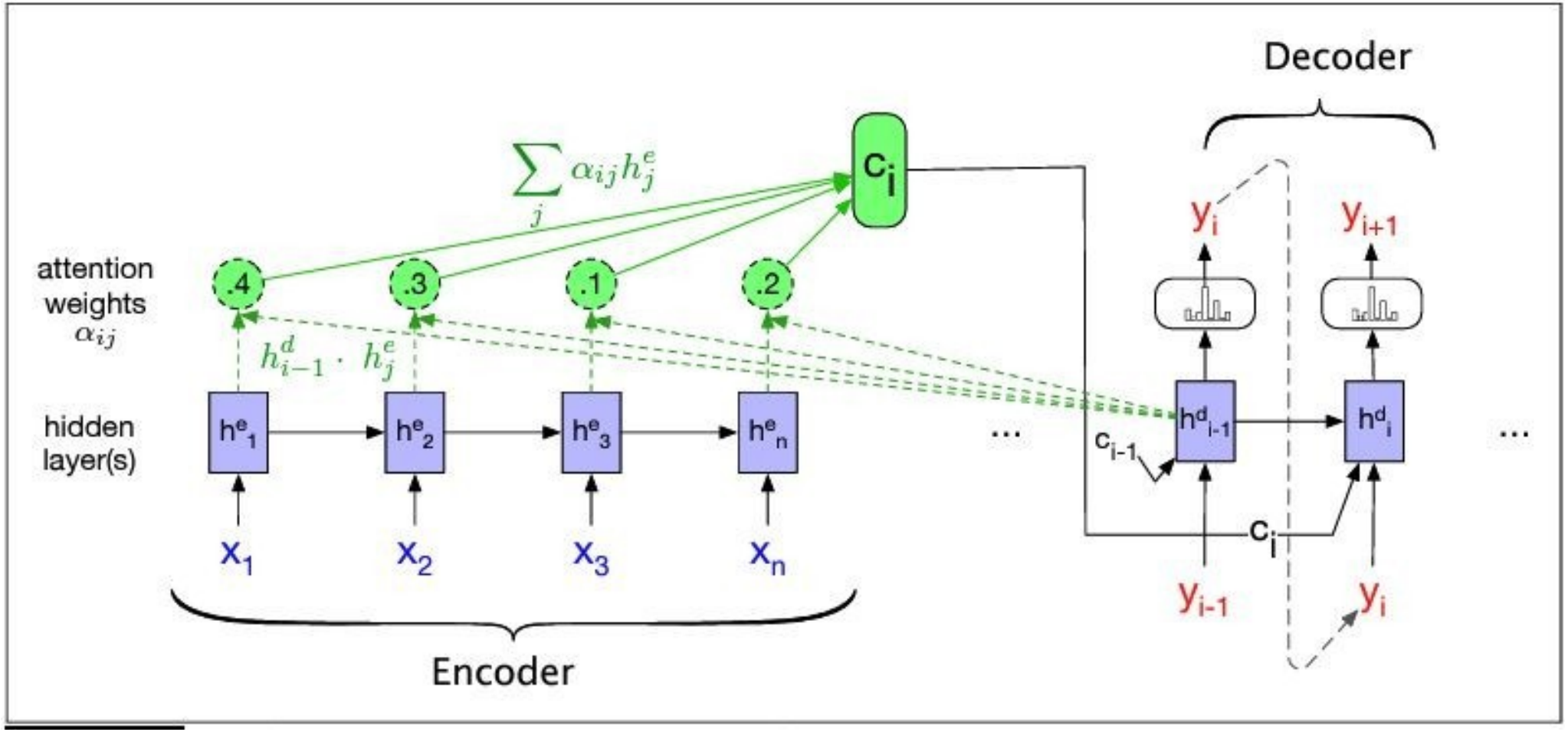
The final piece: the bottleneck problem in RNNs

Why is this a bottleneck?
Because:
You are forcing:
A long sentence → squeeze into one vector → reconstruct another long sentence
Information loss happens.



Problems with this architecture?

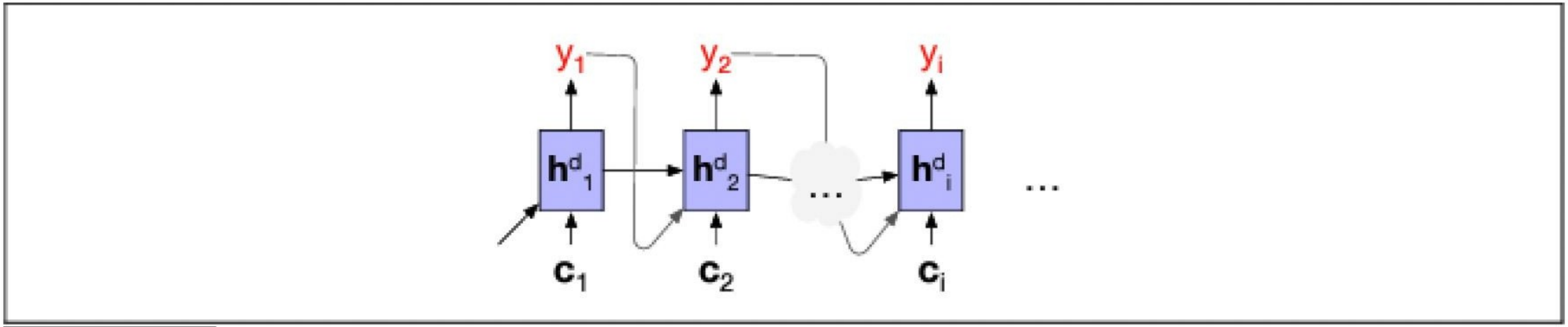
Encoder-decoder with attention



Source: https://web.stanford.edu/~jurafsky/slp3.

Neural Machine Translation by Jointly Learning to Align and Translate https://arxiv.org/abs/1409.0473

Encoder-decoder with attention



The attention mechanism allows each hidden state of the decoder to see a different, dynamic context, which is a function of all the encoder states

Source: https://web.stanford.edu/~jurafsky/slp3.

Attention: In Equations

Encoder:

$$h_t = f(W_e x_t + U_e h_{t-1})$$

Decoder:

$$s_t = f(W_d y_{t-1} + U_d s_{t-1})$$

Attention:

$$\alpha_{t,i} = \text{softmax}(s_t^\top h_i)$$

$$c_t = \sum_i \alpha_{t,i} h_i$$

Output:

$$\tilde{s}_t = \tanh(W_c[s_t; c_t])$$

$$y_t = \text{softmax}(V\tilde{s}_t)$$

Source: https://web.stanford.edu/~jurafsky/slp3.

Try this problem

Consider a sequence-to-sequence RNN model that uses dot-product attention. The encoder outputs four hidden states, each of dimension 2: [2, 0], [−1, 2], [1, −1], and [0, 1]. At a particular decoding step, the decoder hidden state is [1, −2]. Using dot-product attention over the encoder hidden states, compute the context vector that will be used as input to the decoder's next hidden state.

Attention is quite helpful

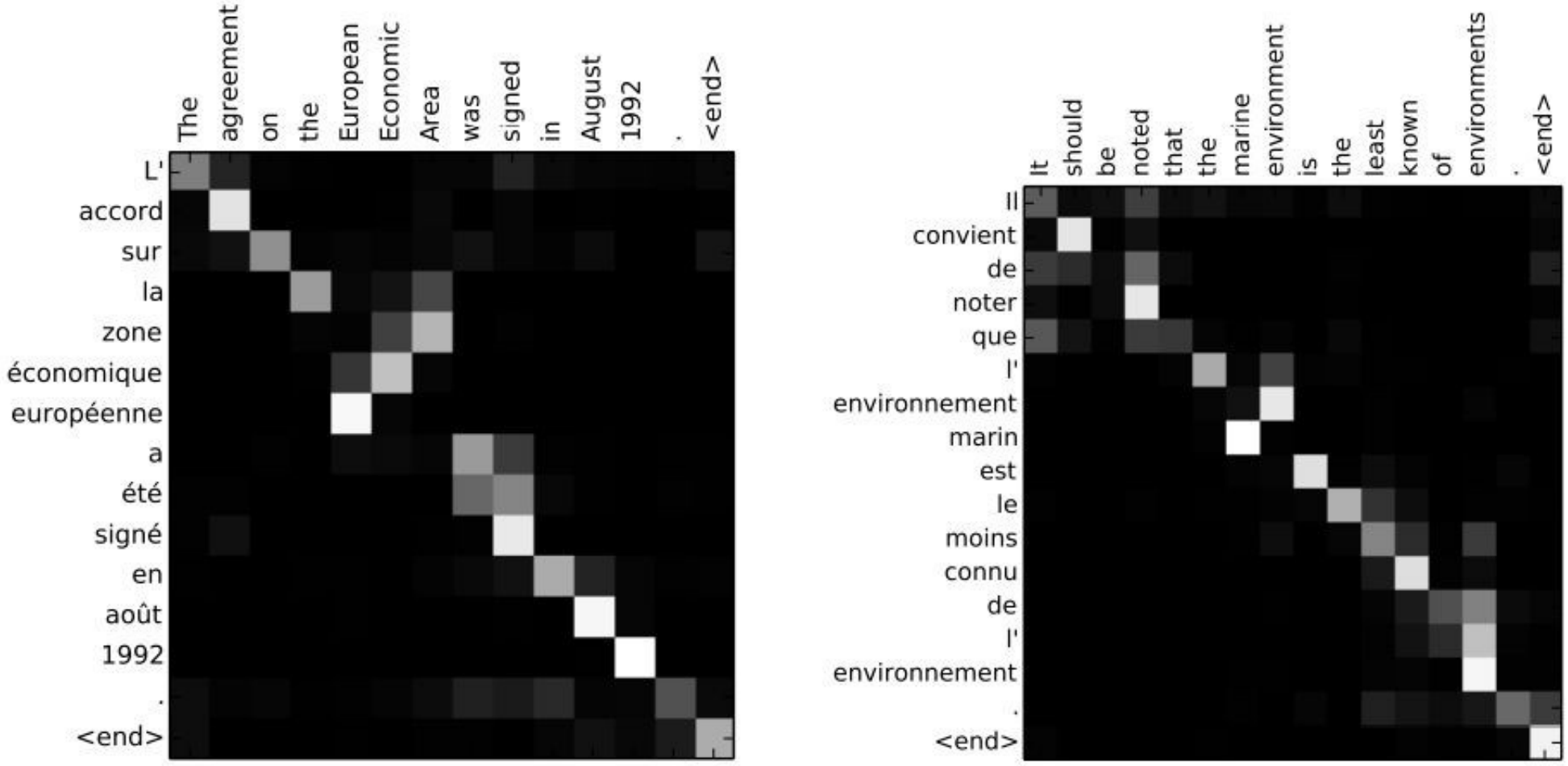
Attention improves NMT performance

It is useful to allow decoder to focus on certain parts of the source

Attention provides some interpretability

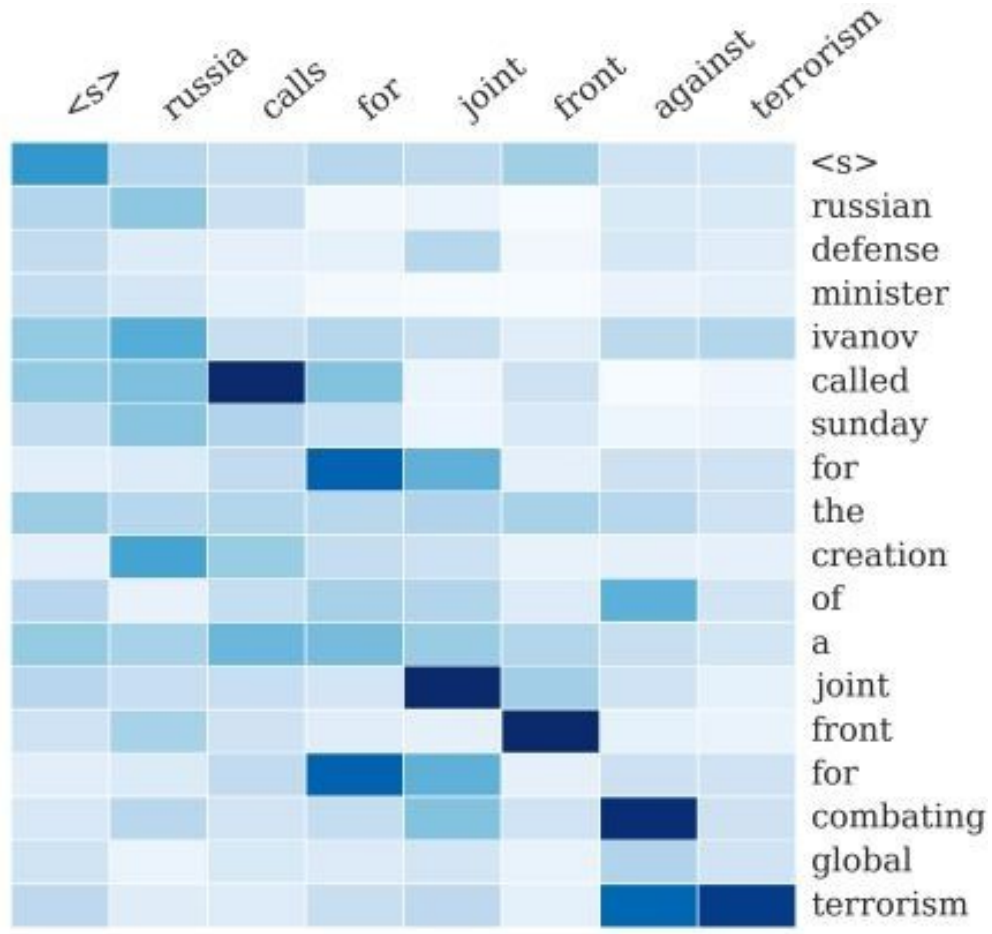
- By inspecting attention distribution, we can see what the decoder was focusing on
- We get alignment for free even if we never explicitly trained an alignment system

Example: Machine Translation



Neural Machine Translation by jointly learning to align and Translate, ICLR 2015

Example: Machine Translation



A Neural Attention Model for Sentence Summarization, EMNLP 2015

Effect of vanishing gradient on RNN-LM

- **LM task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____*

- To learn from this training example, the RNN-LM needs to model the dependency between *"tickets"* on the 7th step and the target word *"tickets"* at the end.

- But if the gradient is small, the model can't learn this dependency
  - So, the model is unable to predict similar long-distance dependencies at test time



- **Syntactic recency:** *The writer of the books is* (correct)



- **Sequential recency:** *The writer of the books are* (incorrect)

How to fix vanishing gradient problem?

- The main problem is that it is too difficult for the RNN to learn to preserve information over many timesteps.
- In a vanilla RNN, the hidden state is constantly being rewritten

$$h^{(t)} = tanh(Uh^{(t-1)} + Wx^{(t)})$$

- How about better RNN units?

Material for reference: https://www.youtube.com/watch?v=9TFnjJkfqmA https://www.youtube.com/watch?v=fyc0Jzr74y4&list=PLoROMvodv4rOaMFbaqxPDoLWjDaRAdP9D&index=5

Using Gates for better RNN Units

- The gates are also vectors
- On each timestep, each element of the gates can be open (1), close (0) or somewhere in-between.
- The gates are dynamic: their value is computed based on the current context.

Two famous architectures

GRUs, LSTMs

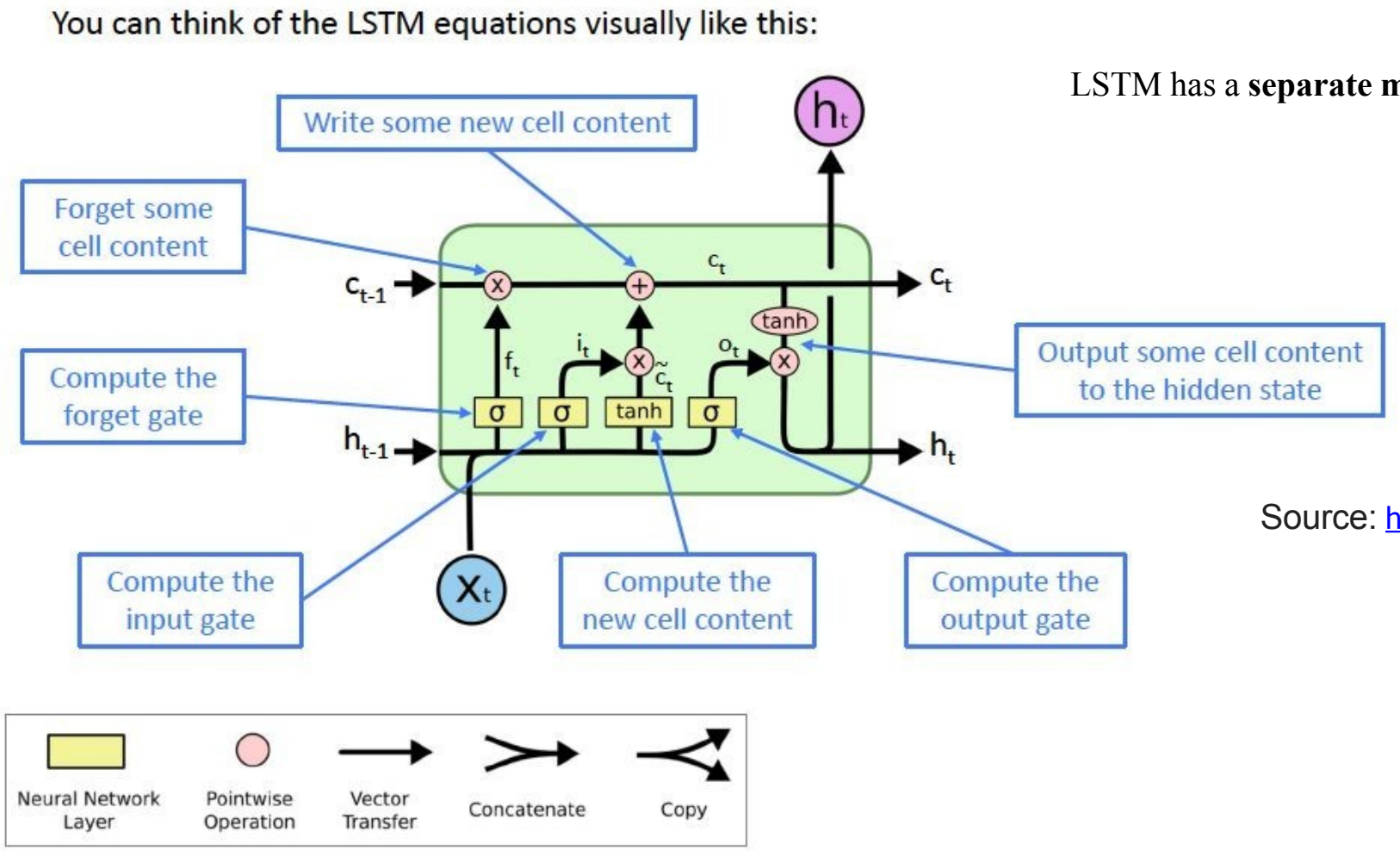
Long Short Term Memory (LSTM)

On step t, there is a hidden state $\boldsymbol{h}^{(t)}$ and a cell state $\boldsymbol{c}^{(t)}$

- Both are vectors length n
- The cell stores long-term information
- The LSTM can read, erase, and write information from the cell

The selection of which information is erased/written/read is controlled by three corresponding gates (gates are calculated things whose values are probabilities)

Long Short Term Memory (LSTM)



LSTM has a **separate memory cell (horizontal line on top)**

Source: https://web.stanford.edu/class/cs224n/

Long Short Term Memory (LSTM): In Equations

Forget Gate

Controls what is kept vs forgotten from the context

$$f_t = \sigma(U_f h_{t-1} + W_f x_t)$$

Input Gate

Controls what parts of new cell content are written to the context

$$i_t = \sigma(U_i h_{t-1} + W_i x_t)$$

New Cell content: $g_t = tanh(U_g h_{t-1} + W_g x_t)$

New Context Vector: $c_t = i_t \odot g_t + f_t \odot c_{t-1}$

Long Short Term Memory (LSTM): In Equations

Output Gate

Controls what part of context are output to hidden state

$$o_t = \sigma(U_o h_{t-1} + W_o x_t)$$

New Hidden State: $h_t = o_t \odot tanh(c_t)$

How does LSTM solve Vanishing Gradient?

Why LSTM Is Better at Remembering Than RNN

LSTM is built in a way that helps the network **keep important information for a long time**.

If the **forget gate** tells the model to “remember everything,” then the memory inside the LSTM **does not change much** and the information can stay for many time steps.

In a normal RNN, there is **no special memory control**. The network must learn weights that keep information by itself, which is very difficult. Because of this, important information often **fades away** as time passes.

LSTM can protect important information using gates, while a simple RNN cannot, so LSTM remembers long-term information better.

Try this problem

Assume you are using a Bi-LSTM model for language identification in code-switched text. Each word is represented by a 60-dimensional embedding. The forward and backward LSTMs each use a hidden state size of 32. There are 4 possible language tags per word. Ignoring bias terms, how many total trainable parameters does the model have?

GRU

What is GRU?
GRU is a type of Recurrent Neural Network (RNN) designed to handle sequential data while avoiding the problem of forgetting important past information too quickly.

Main Components
Hidden State (Memory)

- Stores information from previous time steps
- Acts as the network's short-term and long-term memory combined

Reset Gate
Purpose: Controls how much past information should be forgotten

- Decides which parts of the previous memory are important
- Helps the model ignore irrelevant old data
- Useful when the current input is more important than the past

GRU

Update Gate

Purpose: Controls how much new information should be stored

- Decides how much of the new candidate information should replace old memory
- Helps preserve important long-term patterns
- Prevents unnecessary overwriting of useful information

Candidate Activation

- Represents new information created from the current input
- Modified using the reset gate
- Acts as a possible update to memory

Final Output

- Produced from the updated hidden state
- Used for predictions such as classification, translation, or forecasting

GRU

Update Gate Equation

Controls how much old memory is kept and how much new information is added.

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

Reset Gate Equation

Controls how much past information is forgotten.

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

Candidate Hidden State Equation

Generates new information using current input and filtered past memory.

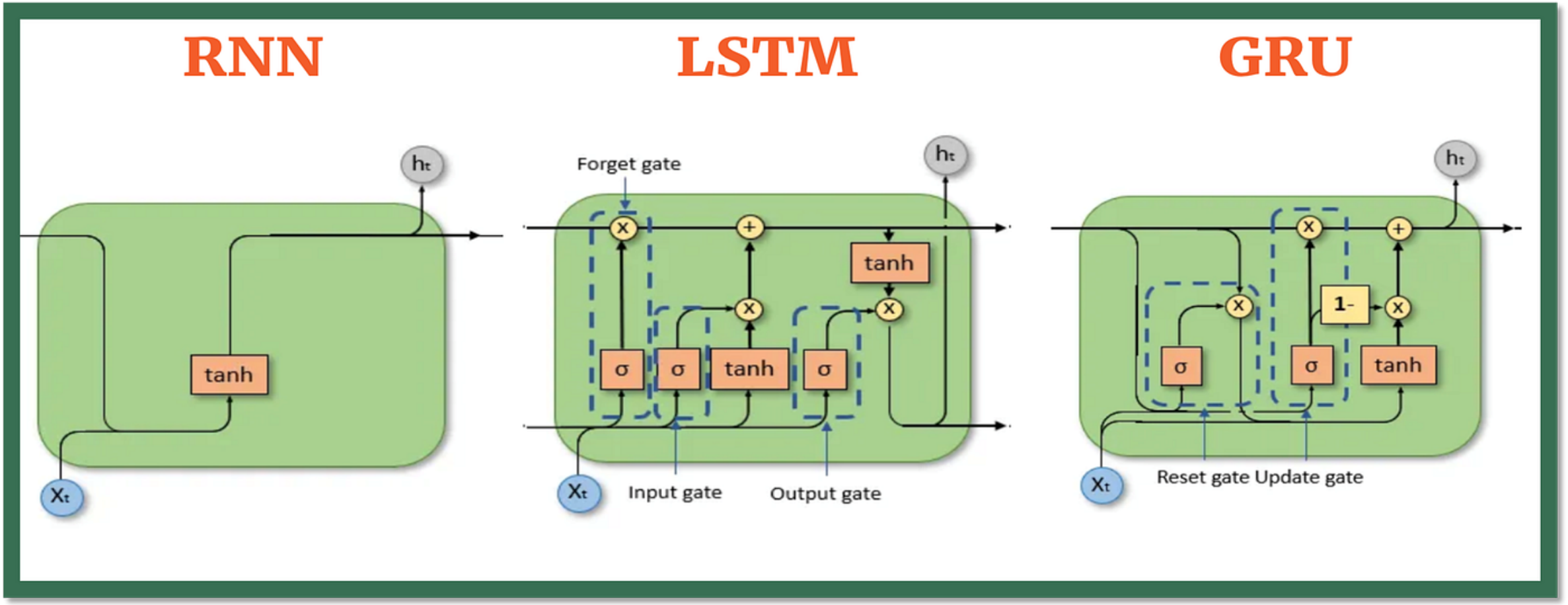
$$h_t = tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$

Final Hidden State Update Equation

Combines old memory and new candidate information to form the updated hidden state.

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot h_t$$

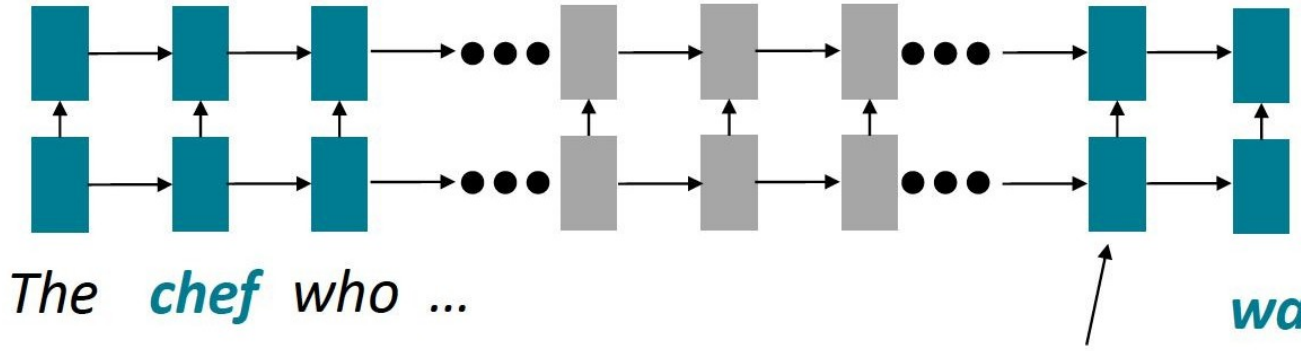
GRU


RNN
LSTM
GRU
h_t
tanh
X_t
Forget gate
σ
tanh
Input gate
Output gate
1-
Reset gate
Update gate

Issues with recurrent models

O(sequence length) steps for distant word pairs

- Hard to learn long-distance dependencies (gradient problems!)
- Linear order of words is "baked in"; not the right way to think about sentences



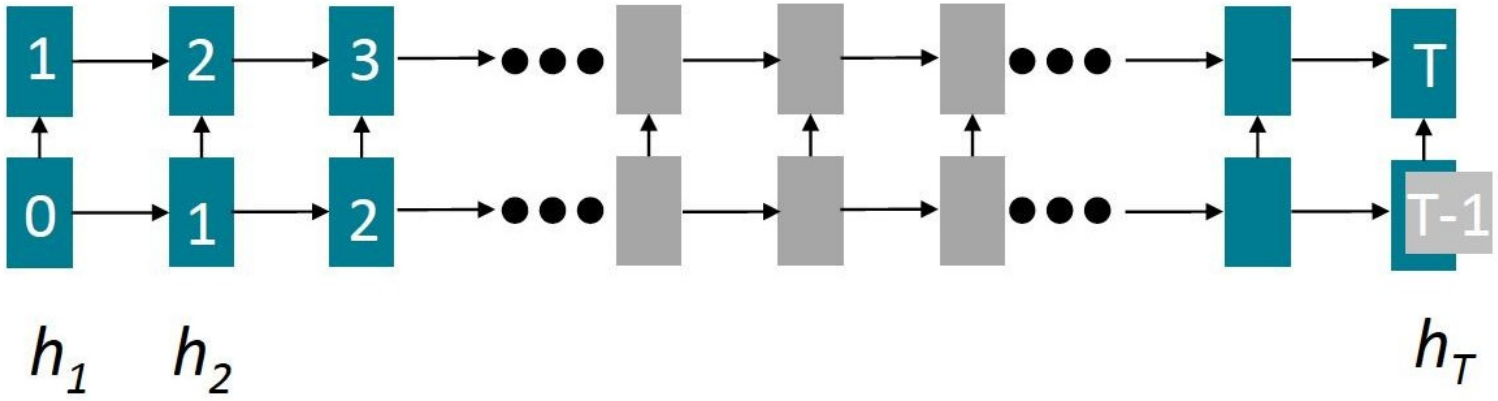
Info of ***chef*** has gone through O(sequence length) many layers!

https://web.stanford.edu/class/cs224n/

Issues with recurrent models

Lack of parellilizability

- Future RNN hidden states can't be computed in full before past RNN hidden states have been computed



Inhibits training on very large datasets!

Numbers indicate min # of steps before a state can be computed

https://web.stanford.edu/class/cs224n/

LSTMs Revamped: xLSTMs

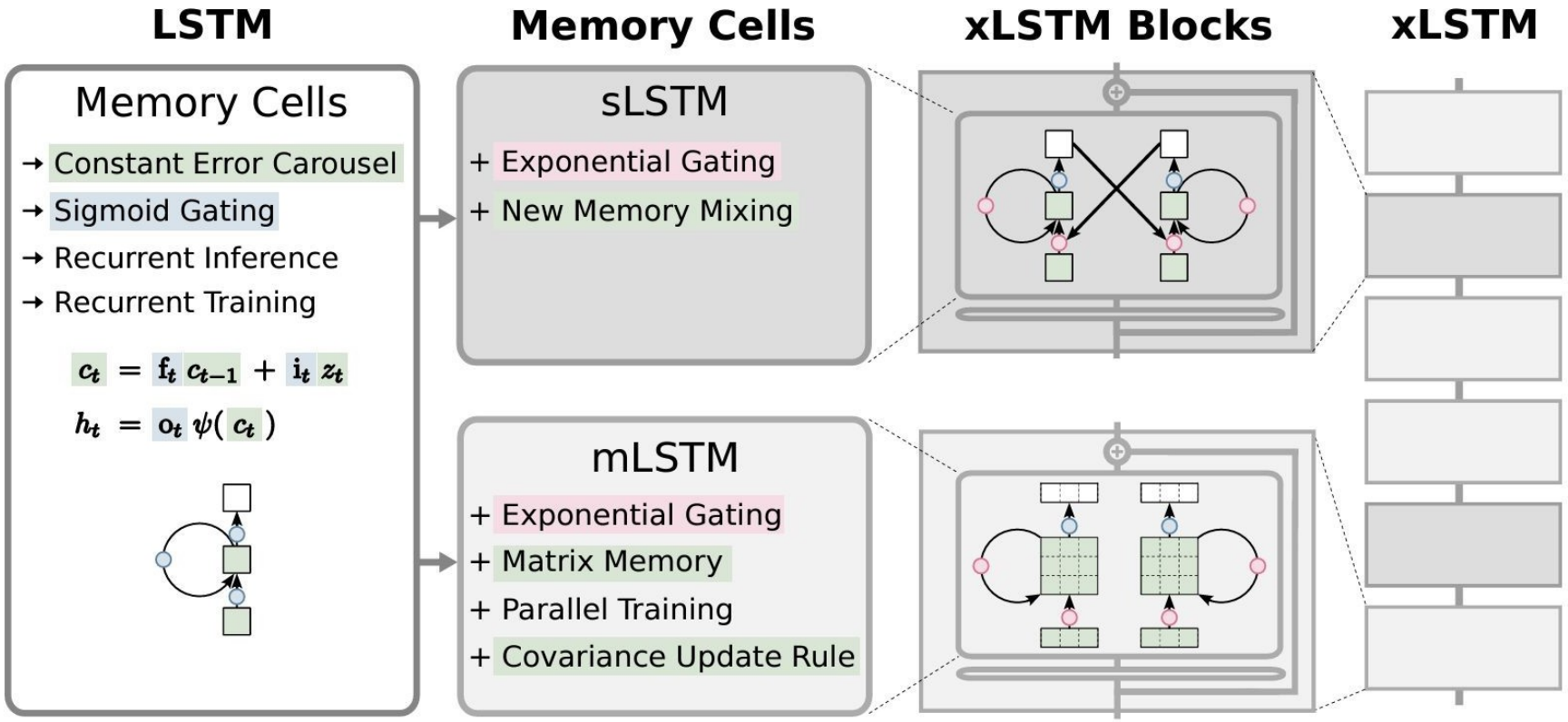


Figure 1: The extended LSTM (xLSTM) family. From left to right: 1. The original LSTM memory cell with constant error carousel and gating. 2. New sLSTM and mLSTM memory cells that introduce exponential gating. sLSTM offers a new memory mixing technique. mLSTM is fully parallelizable with a novel matrix memory cell state and new covariance update rule. 3. mLSTM and sLSTM in residual blocks yield xLSTM blocks. 4. Stacked xLSTM blocks give an xLSTM architecture.

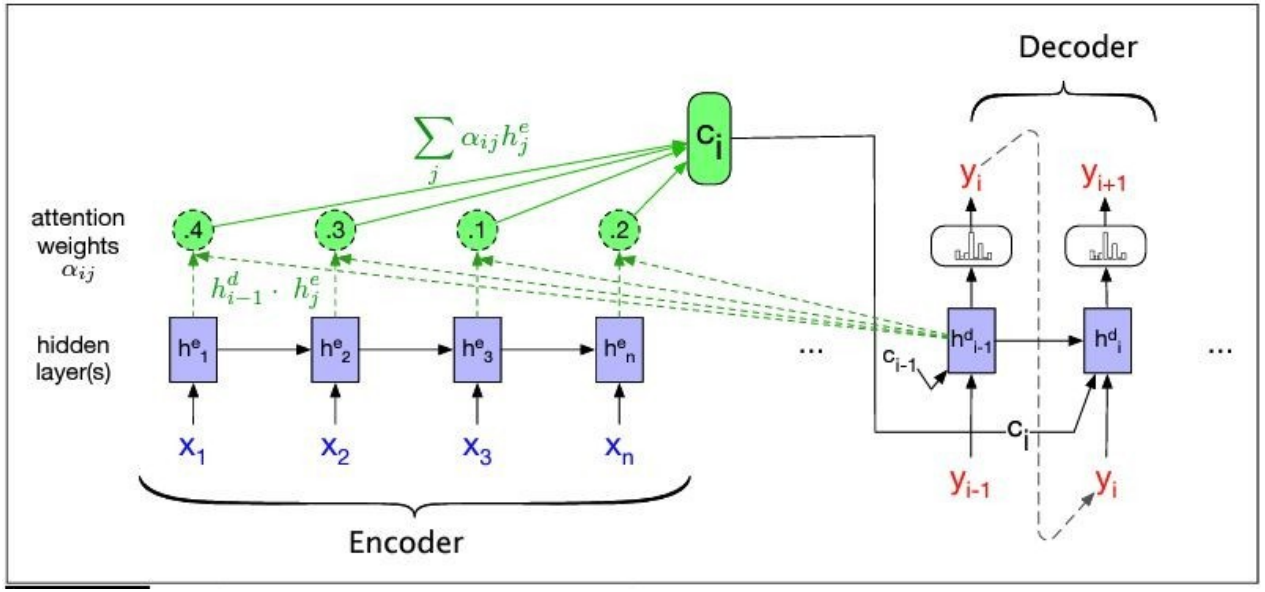
xLSTM: Extended Long Short-Term Memory. https://arxiv.org/pdf/2405.04517

Low-resource NMT

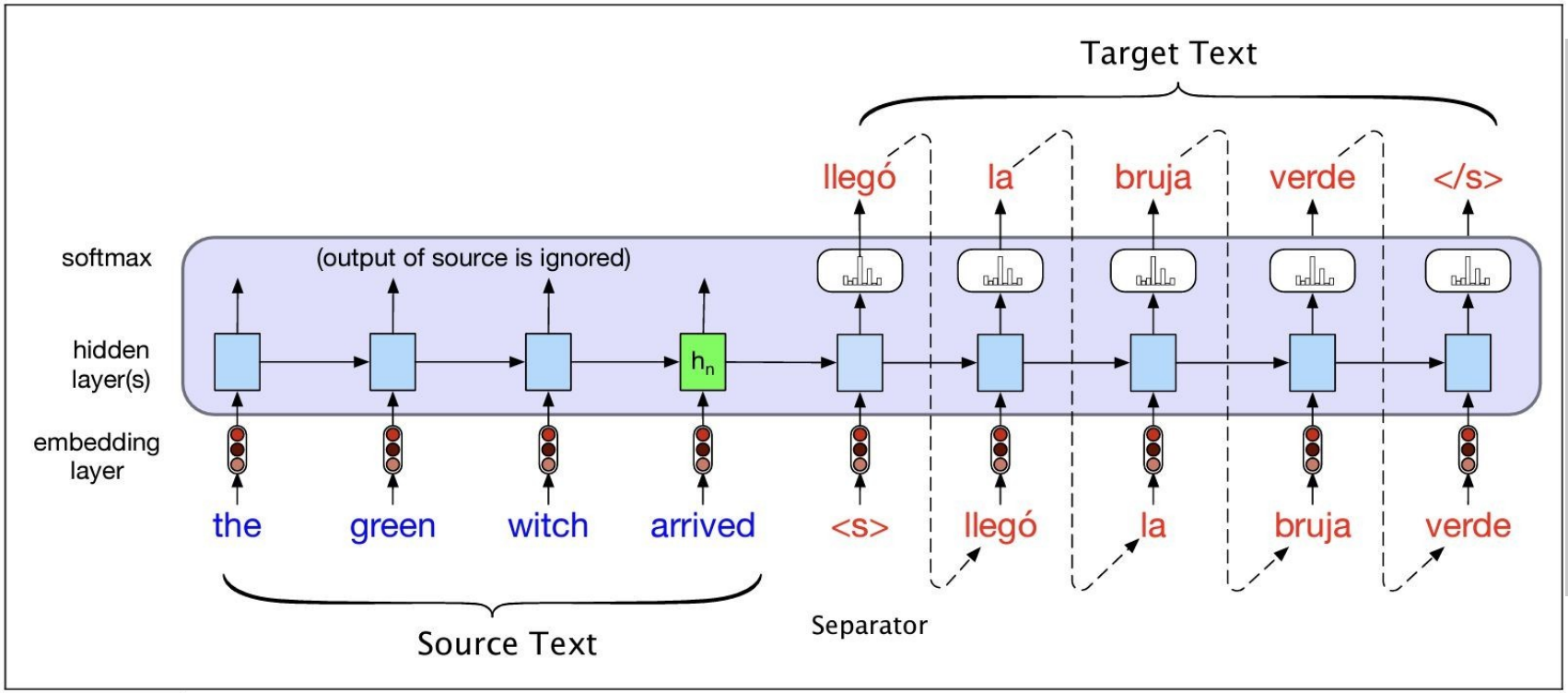
1. Transfer Learning
Train on **high-resource language pairs first**, then fine-tune on low-resource data.
Reuses learned language knowledge.
2. Multilingual NMT
Train **one model on many languages together**.
Low-resource languages benefit from shared representations.
3. Back Translation
Translate **monolingual target text back to source** to create fake parallel data.
Increases training data.
4. Data Augmentation
Create more data using **paraphrasing, noise injection, or word replacement**.
Improves robustness.
5. Pivot-Based Translation
Translate via a **third language (pivot)** when direct data is scarce.
Example: A → English → B.
6. Unsupervised NMT
Uses **only monolingual data**, no parallel corpus.
Works when bilingual data is unavailable.

Try this Problem

Consider a sequence-to-sequence RNN model with attention for machine translation from Spanish to French. Suppose the source (Spanish) vocabulary size is 35,000 and the target (French) vocabulary size is 45,000. Both languages use 256-dimensional word embeddings. The encoder hidden state size is 128, and the decoder hidden state size is 256. Ignoring all bias terms, how many trainable parameters does the model contain?



Recap: Encoder-decoder model at inference



Translating a single sentence (inference time) in the basic RNN version of encoder-decoder approach to machine translation. Source and target sentences are concatenated with a separator token in between, and the decoder uses context information from the encoder's last hidden state.

How do we generate the tokens in an autoregressive manner?

Source: https://web.stanford.edu/~jurafsky/slp3.

Which words do we generate at each step?

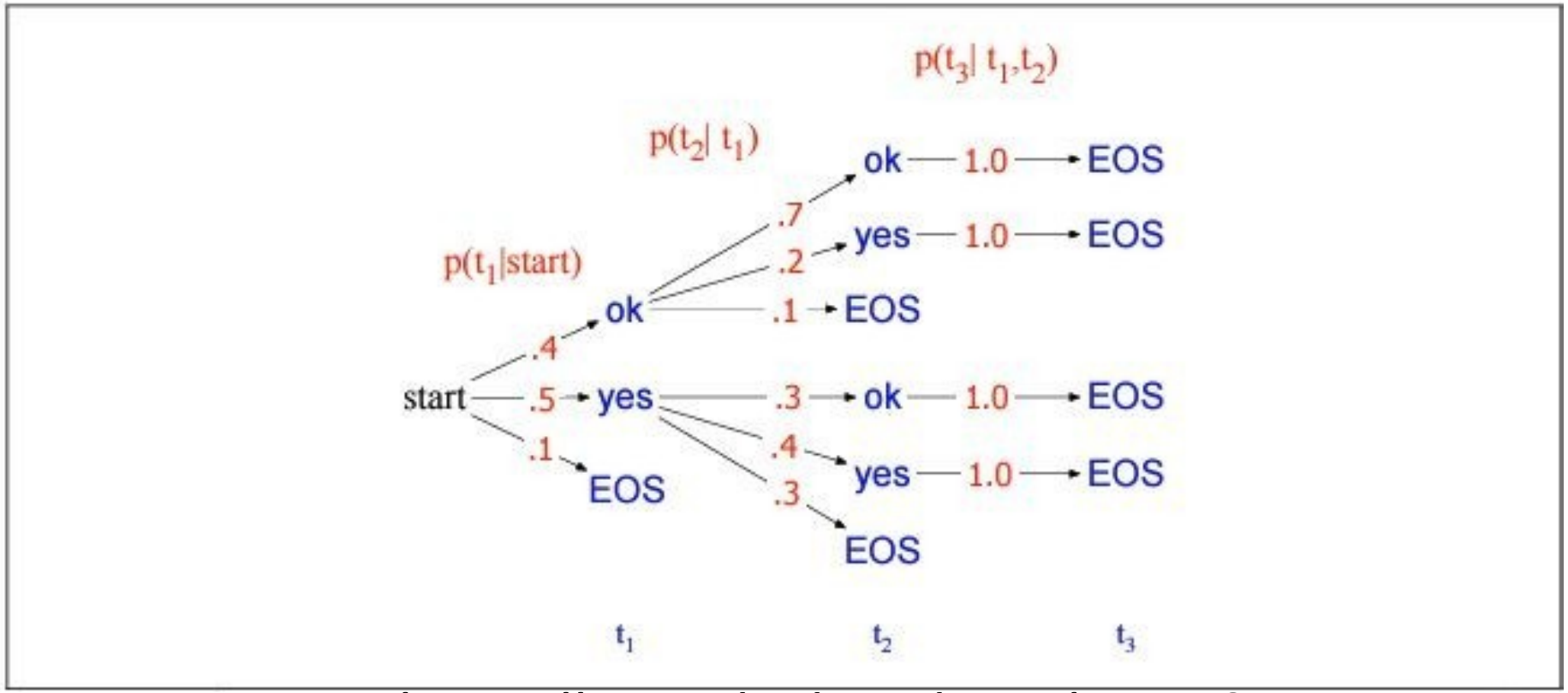
Greedy Decoding

One simple way is to always generate the most likely word given the context

Generating the most likely word given the context is called *greedy decoding*.

$$\hat{w}_t = \arg\max_{w \in V} P(w|w_{<t})$$

What is the issue with greedy decoding?



- *What will greedy decoding choose?*
- *What is globally optimal?*

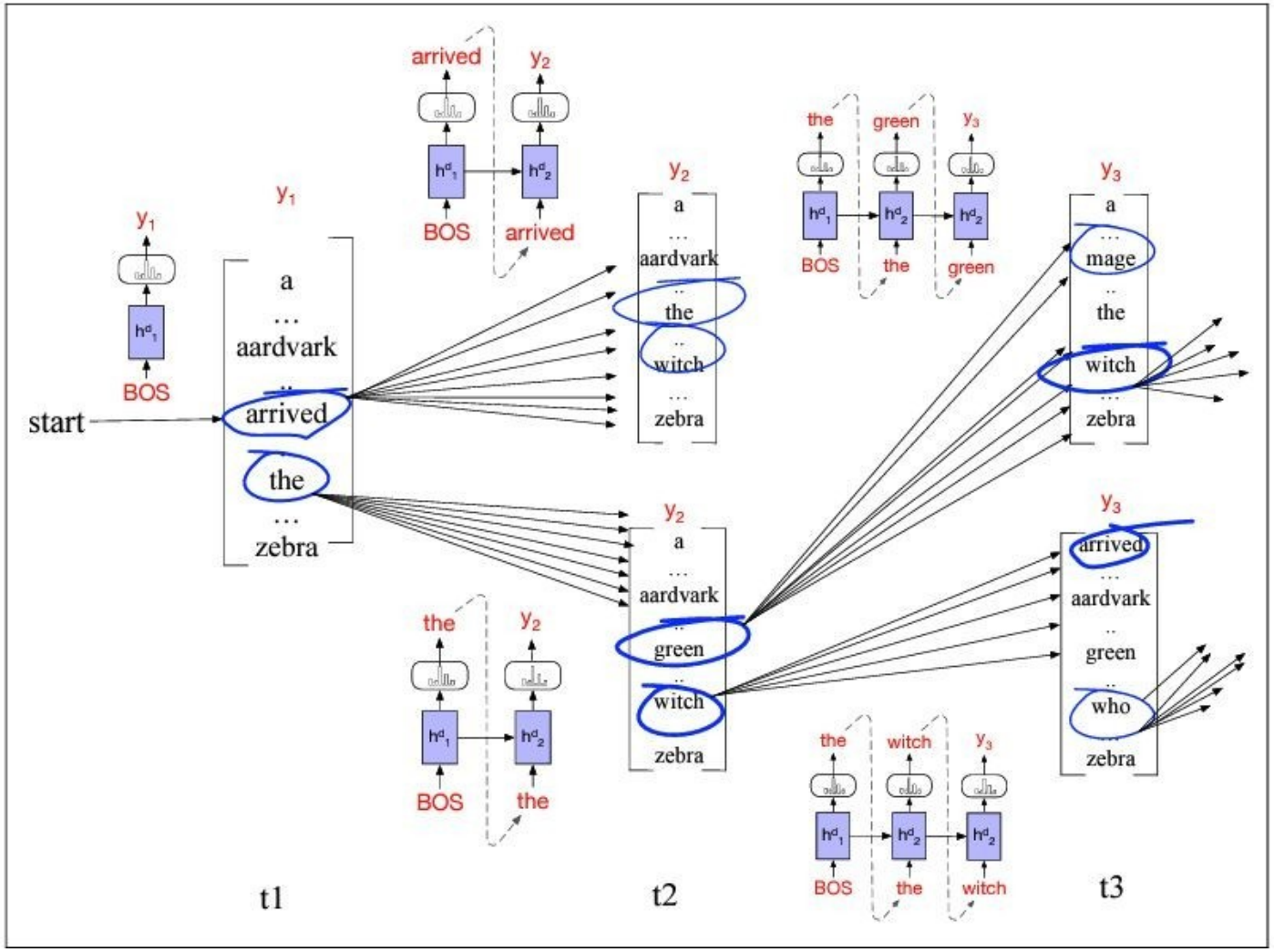
Source: https://web.stanford.edu/~jurafsky/slp3.

Beam Search

Core Idea

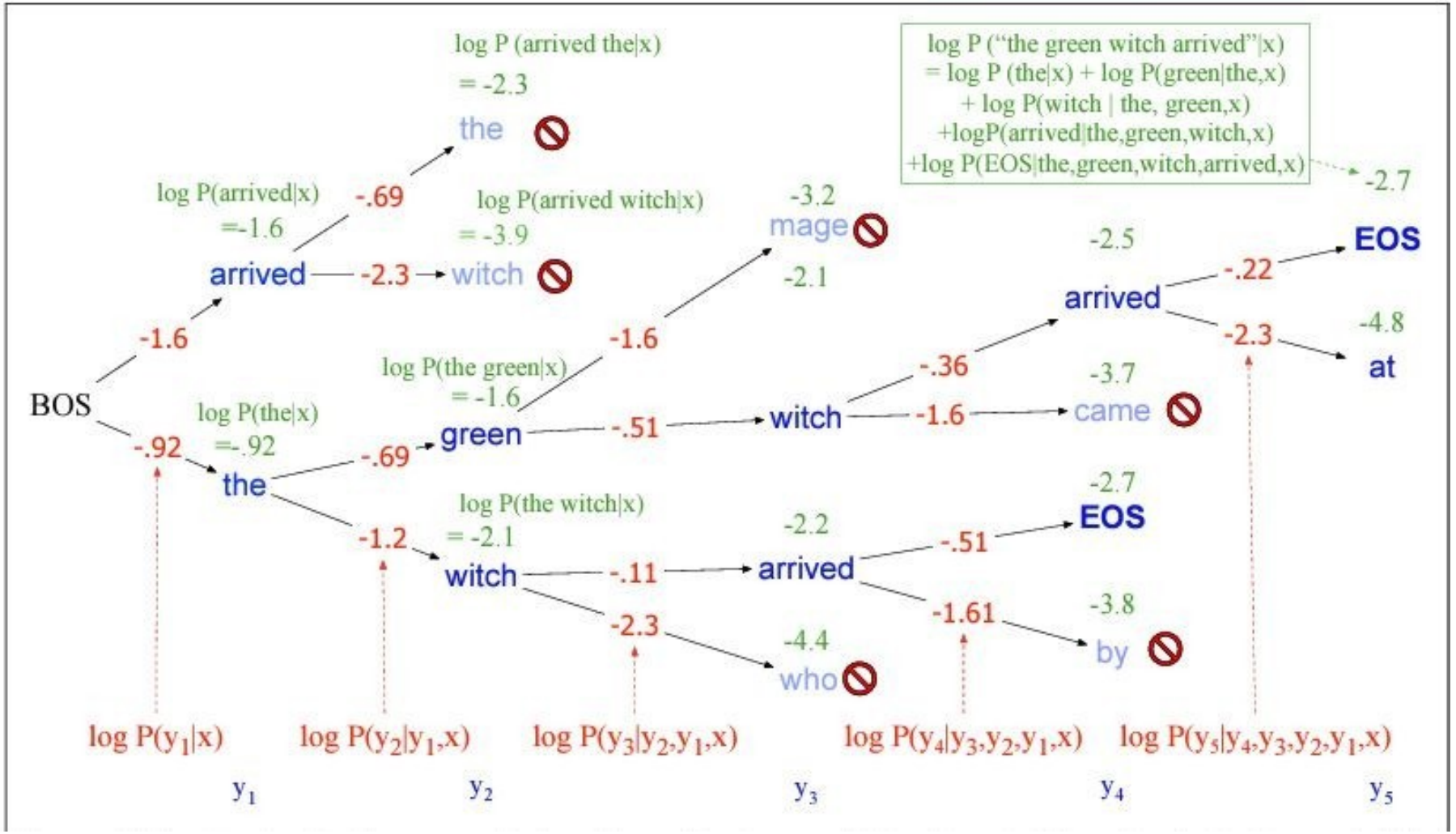
- On each step of the decoder, keep track of the k most probable partial translations (hypotheses)
- k is the beam size / beam width (in practice around 5 to 10)

Beam Search: How does it work?

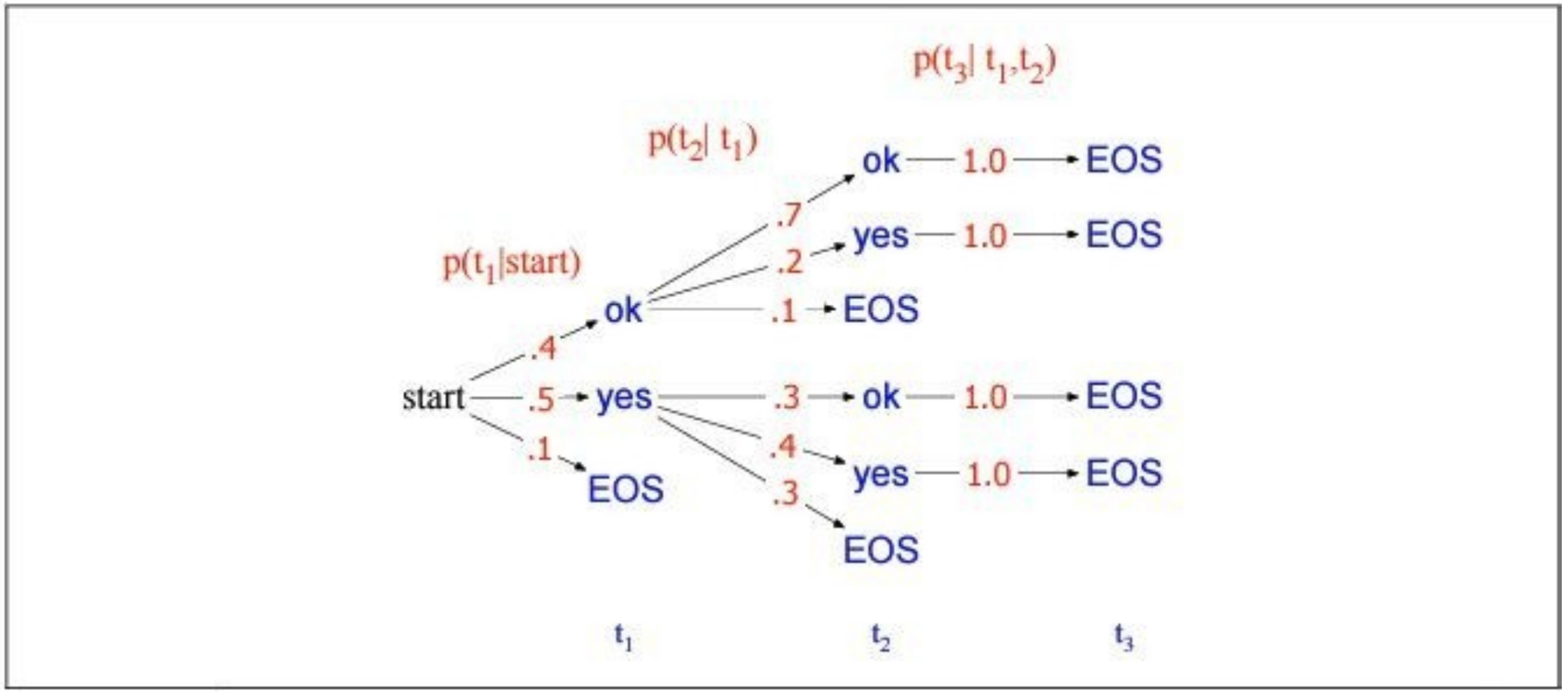


https://web.stanford.edu/~jurafsky/slp3

Beam Search: How does it work?



Beam Search on the prev. example



Take Beam Size = 2. Is the Beam search solution same as the optimal solution?

Beam Search: Stopping Criterion

- In greedy decoding, usually we decode until the model produces an $\langle END \rangle$ token
  - **Example:** $\langle START \rangle$ he hit me with a pie $\langle END \rangle$
- In beam search decoding, different hypotheses may produce $\langle END \rangle$ tokens on different timesteps
  - When a hypothesis produces $\langle END \rangle$, that hypothesis is complete
  - Place it aside and continue exploring other hypotheses via beam search
- Usually we continue beam search until:
  - We reach timestep T (some pre-defined cutoff), or
  - We have at least n completed hypotheses (some pre-defined cutoff)

Beam Search: Finishing up

- We have our list of completed hypotheses
- How to select the top one with the highest score?

Each hypothesis $y_1, \ldots, y_t$ has a score

$$score(y_1, \ldots, y_t) = logP_{LM}(y_1, \ldots, y_t|x) = \sum_{i=1}^{t} logP_{LM}(y_i|y_1, \ldots, y_{i-1}, x)$$

Problem with this

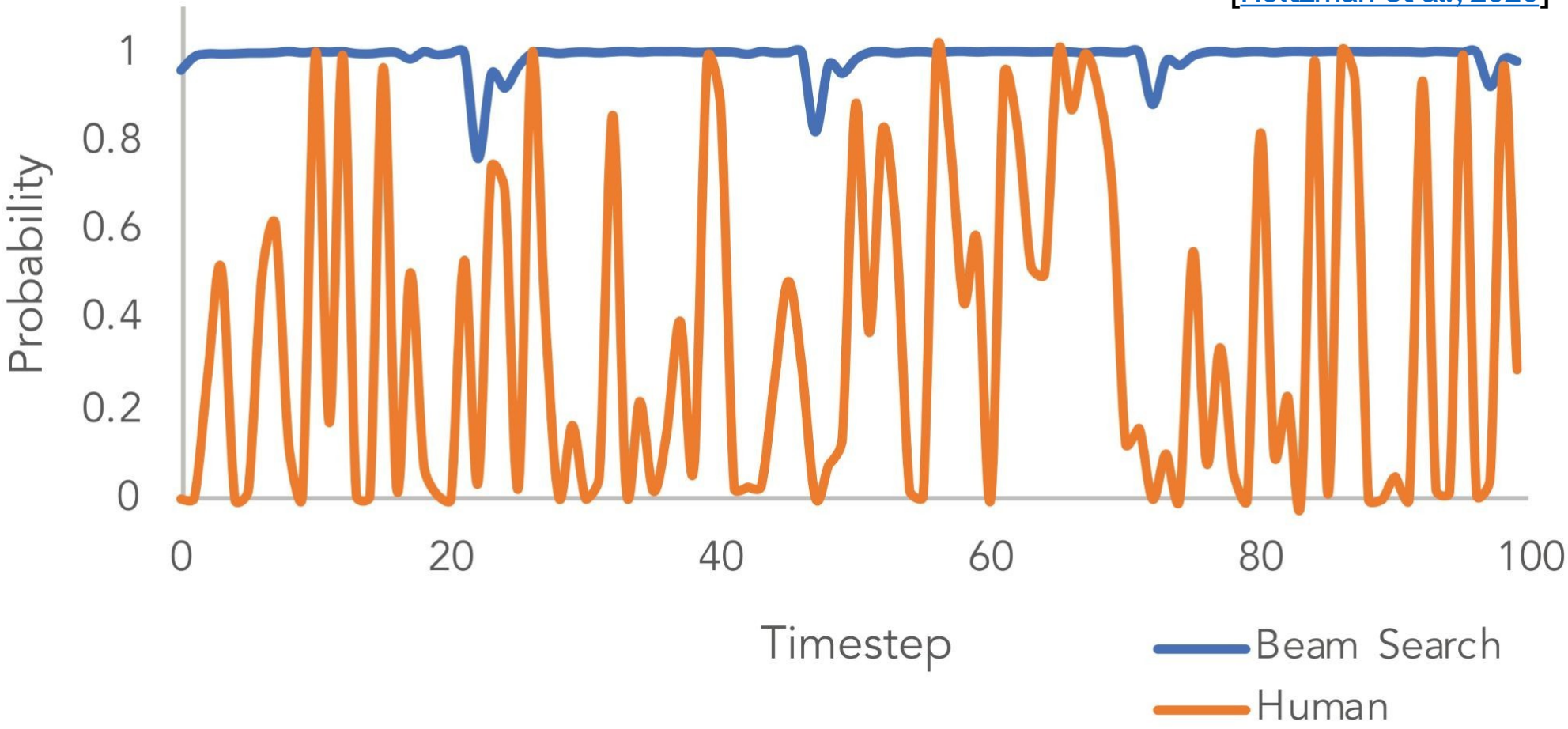
Longer hypotheses have lower scores.

Fix: Normalize the score by length, and use the normalized score to select the top one instead.

$$\frac{1}{t} \sum_{i=1}^{t} logP_{LM}(y_i|y_1, \ldots, y_{i-1}, x)$$

Beam Search Text is Less Surprising

[Holtzman et al., 2020]



https://utah-cs6340-nlp.notion.site/Natural-Language-Processing-bd1a2ca290fc44f69556908ad8d25c70

Greedy and Beam Search are deterministic!

- Greedy decoding as well as Beam Search decoding will give a “deterministic” output
- Other common decoding algorithms involve “sampling”, and bring in some degree of “randomness”

$x \sim p(x) \rightarrow$ choose x by sampling from the distribution $p(x)$

Random Sampling

i $\leftarrow$ 1
$w_i \sim$ p(w)
while w_i != EOS
  i $\leftarrow$ i + 1
  $w_i \sim$ p($w_i \mid w_{<i}$)

Quality vs Diversity trade-off

Various sampling methods enable trading off two important factors in generation: *quality* and *diversity*.

Quality vs Diversity trade-off

- Methods that emphasize the most probable words tend to produce more coherent and accurate generations but also tend to be repetitive and boring
- Methods that give bit more weight to the middle probability words tend to be more creative and diverse, but likely to be incoherent and less factual

Practice Questions

A Skip-gram model is trained with the following settings:

- Vocabulary size = **10,000**
- Embedding dimension = **300**
- Full softmax output layer is used

(a) What is the size of the input embedding matrix?

(b) What is the size of the output weight matrix?

(c) Calculate the total number of trainable parameters (ignore bias terms).

Practice Questions

Consider a Skip-gram model with:

- Vocabulary size = **6,000**
- Embedding dimension = **200**
- Context window size = **2**

(a) What is the dimension of the one-hot encoded input vector?

(b) What is the dimension of the hidden layer (embedding) output?

(c) If a sentence contains 8 words, how many Skip-gram training pairs will be generated?

Practice Questions

Suppose you are given the following vectors and weight matrices for an attention mechanism.

Given:

Decoder hidden state (query):

$$\mathbf{q} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Encoder hidden states (keys and values):

$$\mathbf{k}_1 = \mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{k}_2 = \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Weight matrices and attention vector:

$$W_q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_k = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{v}_a = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

attention score function:

$$\text{score}(\mathbf{q}, \mathbf{k}_i) = \mathbf{v}_a^T \tanh\left(W_q \mathbf{q} + W_k \mathbf{k}_i\right)$$

Tasks:

(a) Compute the attention scores:

$$\text{score}(\mathbf{q}, \mathbf{k}_1), \quad \text{score}(\mathbf{q}, \mathbf{k}_2)$$

(b) Compute the attention weights using softmax:

$$\alpha_i = \frac{e^{\text{score}_i}}{\sum_j e^{\text{score}_j}}$$

(c) Compute the context vector:

$$\mathbf{c} = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2$$